Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Lecture 10:

Avoiding Overfitting and Regularization



Markus Hohle

University of California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

Avoiding Overfitting and Regularization (L1 & L2)





Outline

- **Overfitting**
- **L1 and L2 regularization**
- **A Python Example**
- **Lagrangian Multipliers**

Avoiding Overfitting and Regularization (L1 & L2)



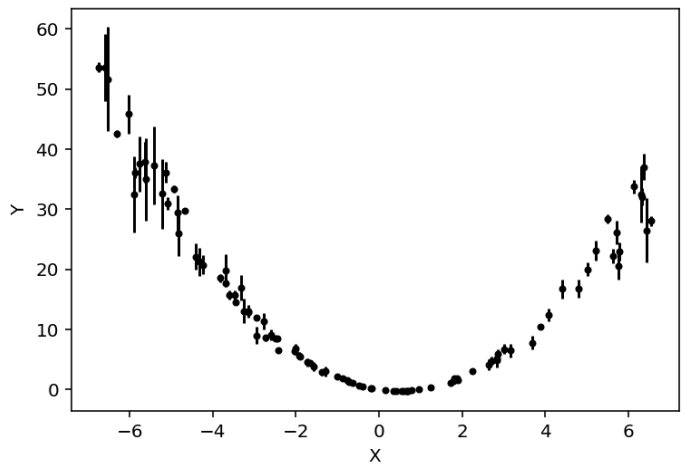


Outline

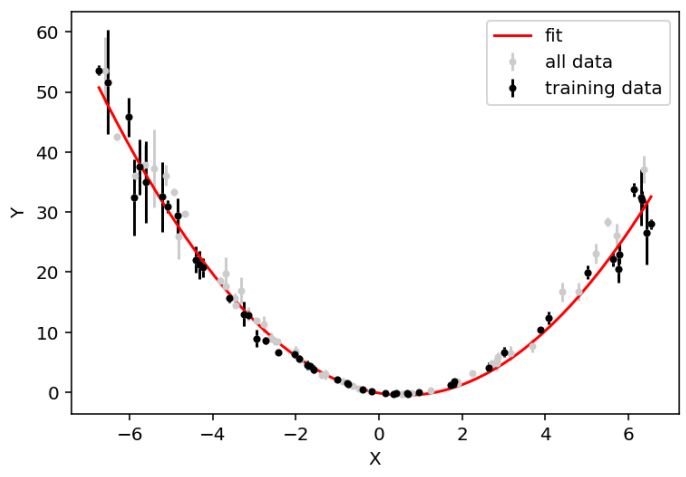
- **Overfitting**
- L1 and L2 regularization
- A Python Example
- Lagrangian Multipliers

Avoiding Overfitting and Regularization (L1 & L2)

- data set that can be fitted with a polynom of 2nd order $y = ax^2 + bx + c$




```
coeff = np.polyfit(X, Y, deg = 2)
poly  = np.poly1d(coeff)
```



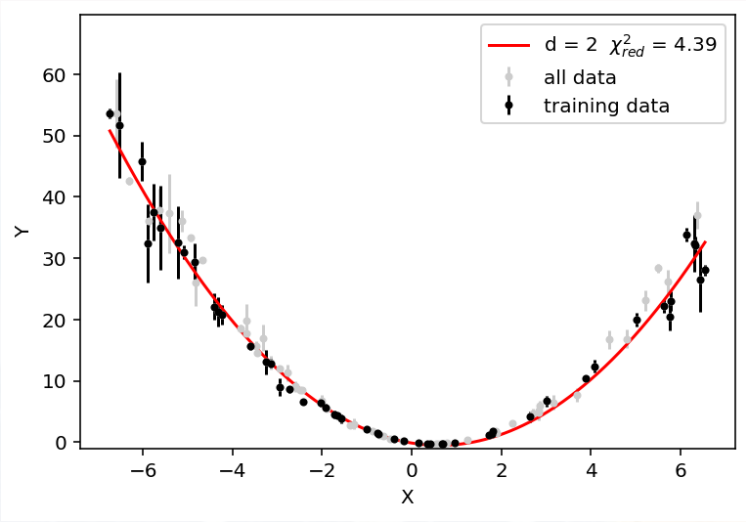
What if we didn't know that deg = 2?

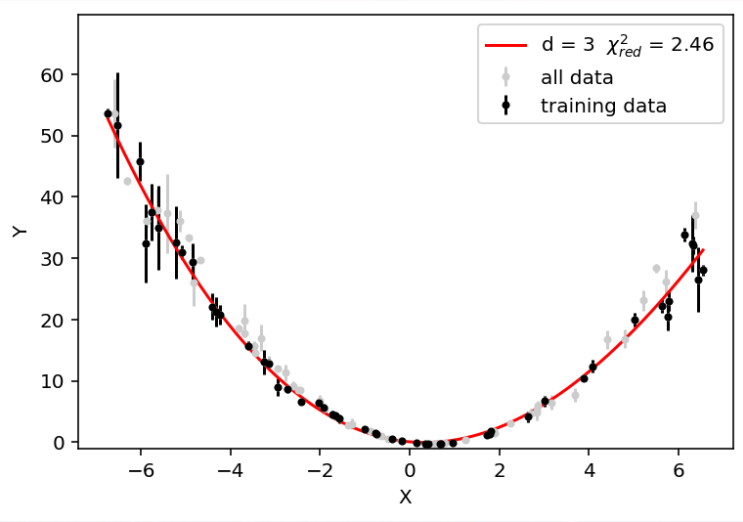
Would we be allowed to make the model more complex (i. e. **increasing deg**) in order to improve χ^2_{red} or MSE?

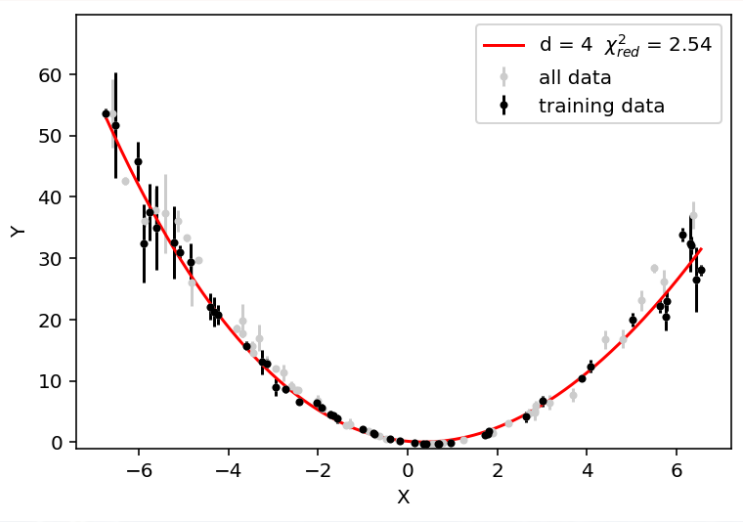
How do we know when to stop?

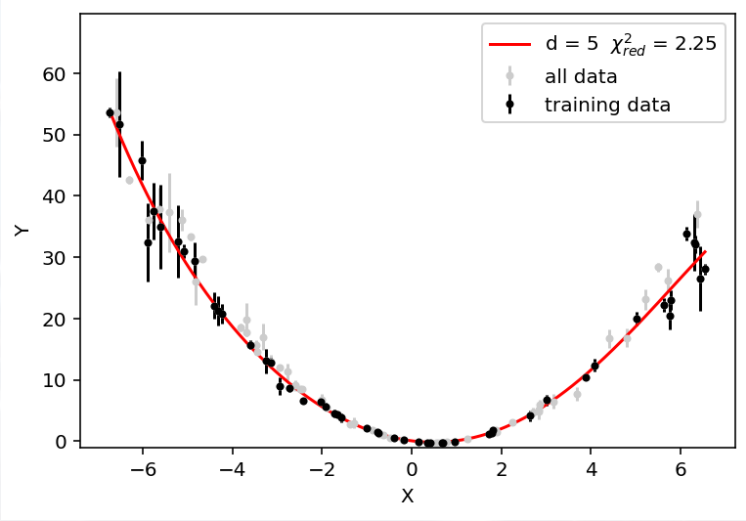
Avoiding Overfitting and Regularization (L1 & L2)

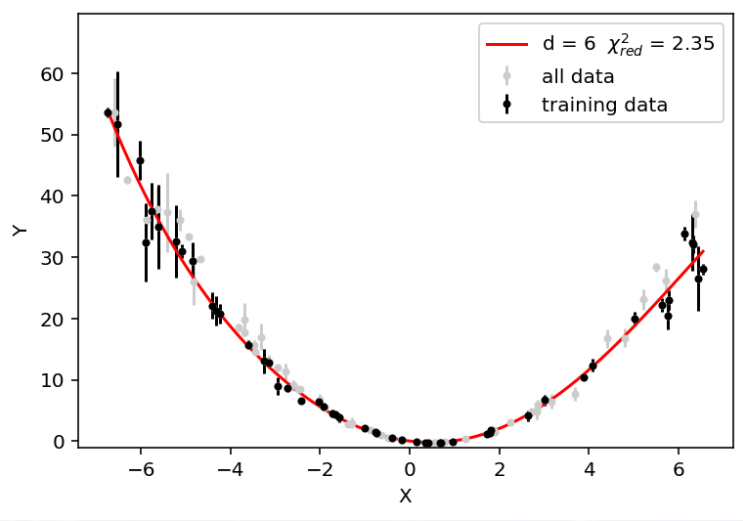
…well, let's try and see what happens:

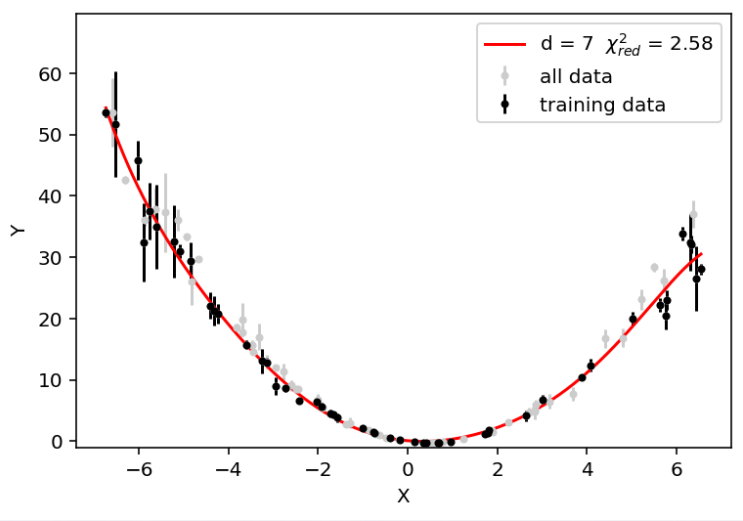






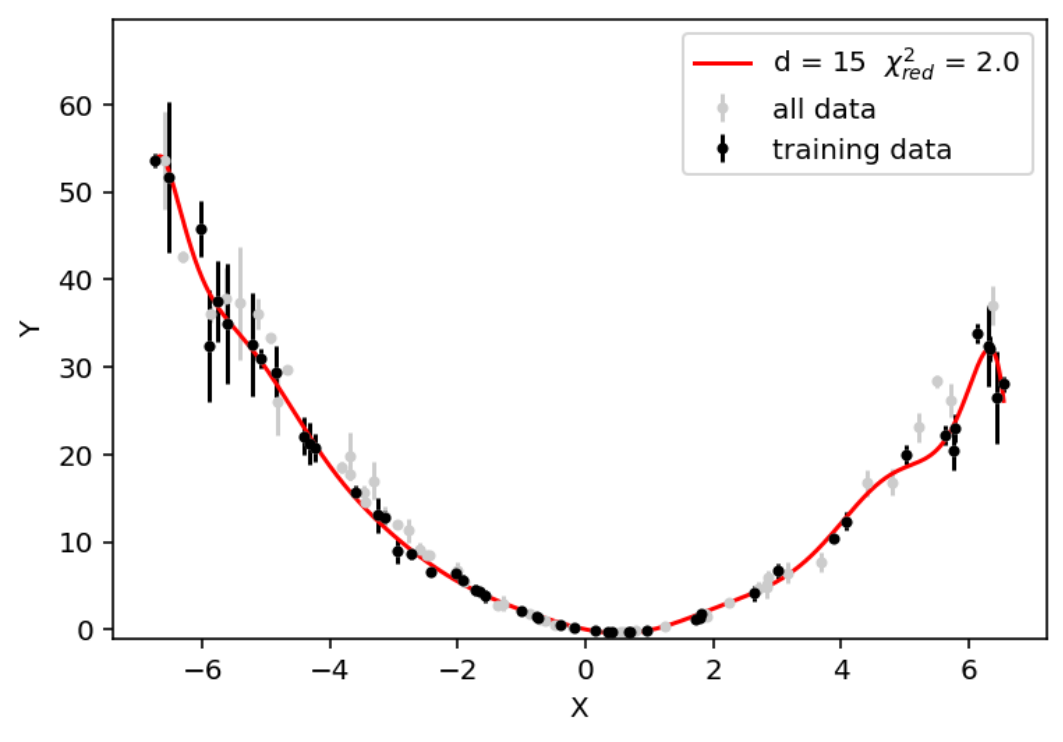


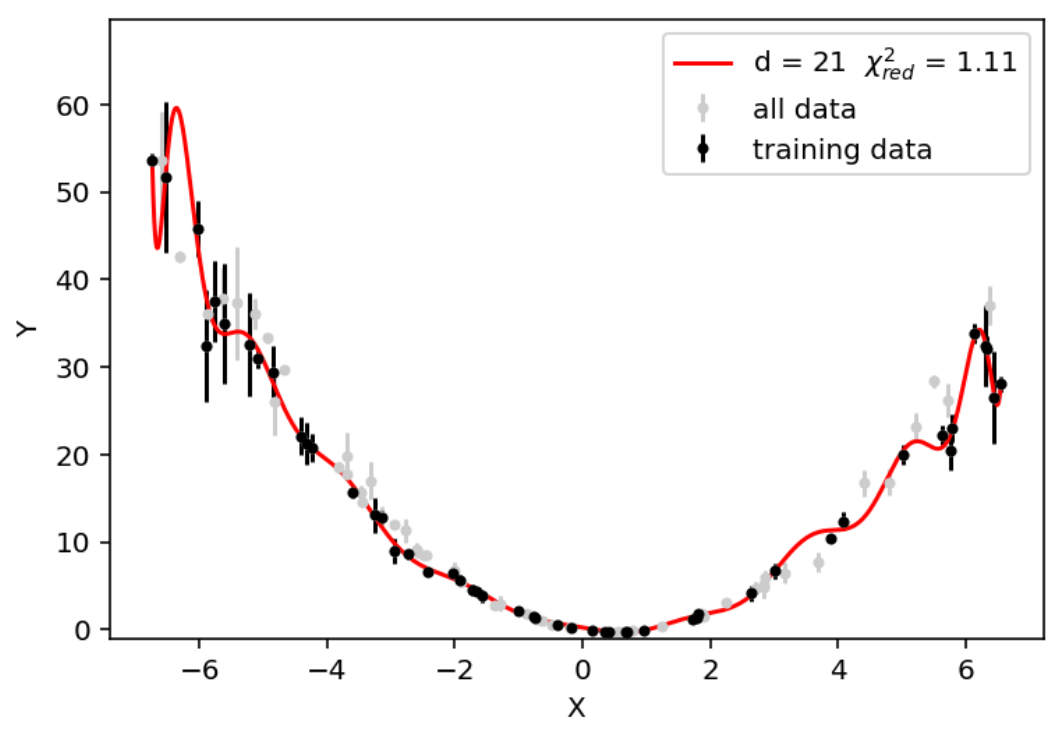




Avoiding Overfitting and Regularization (L1 & L2)

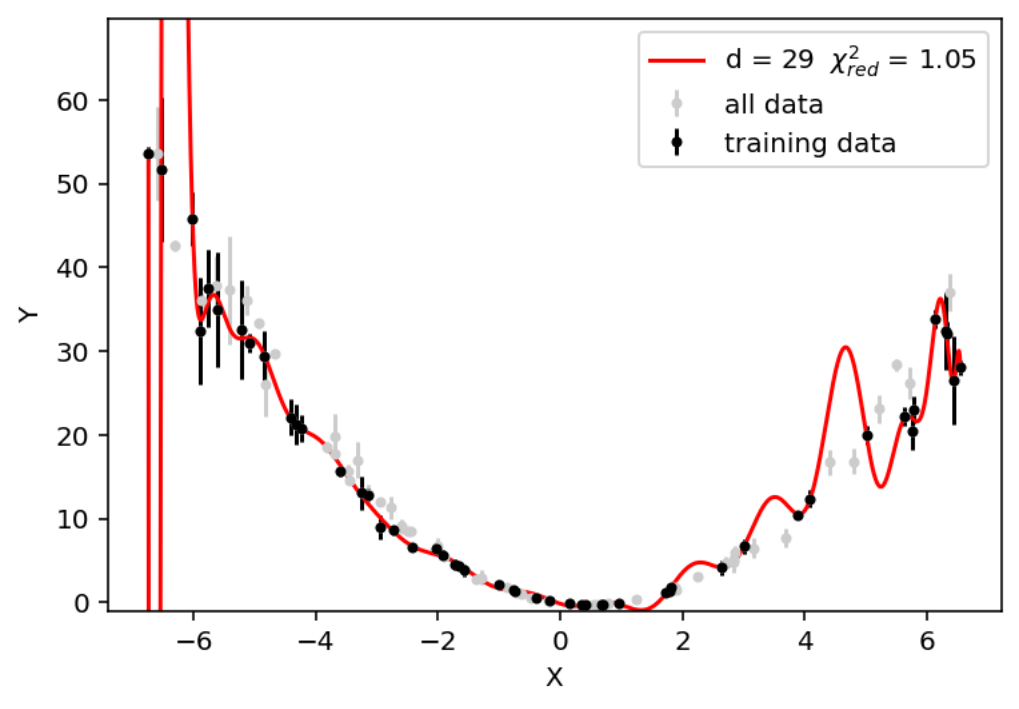
…well, let's try and see what happens:

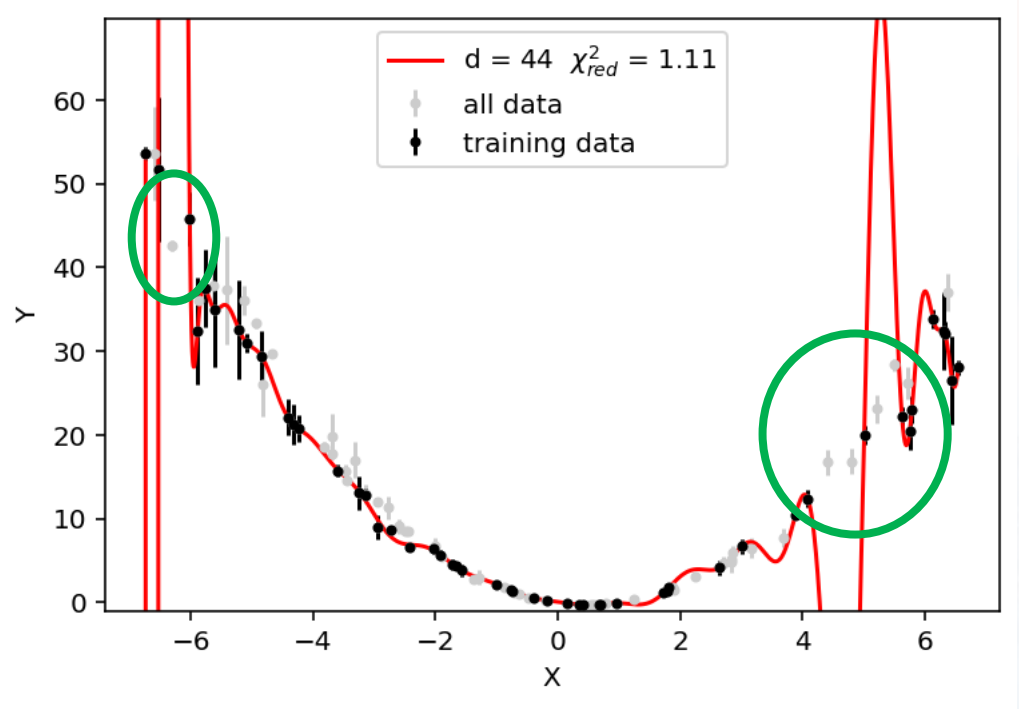




Avoiding Overfitting and Regularization (L1 & L2)

…well, let's try and see what happens:



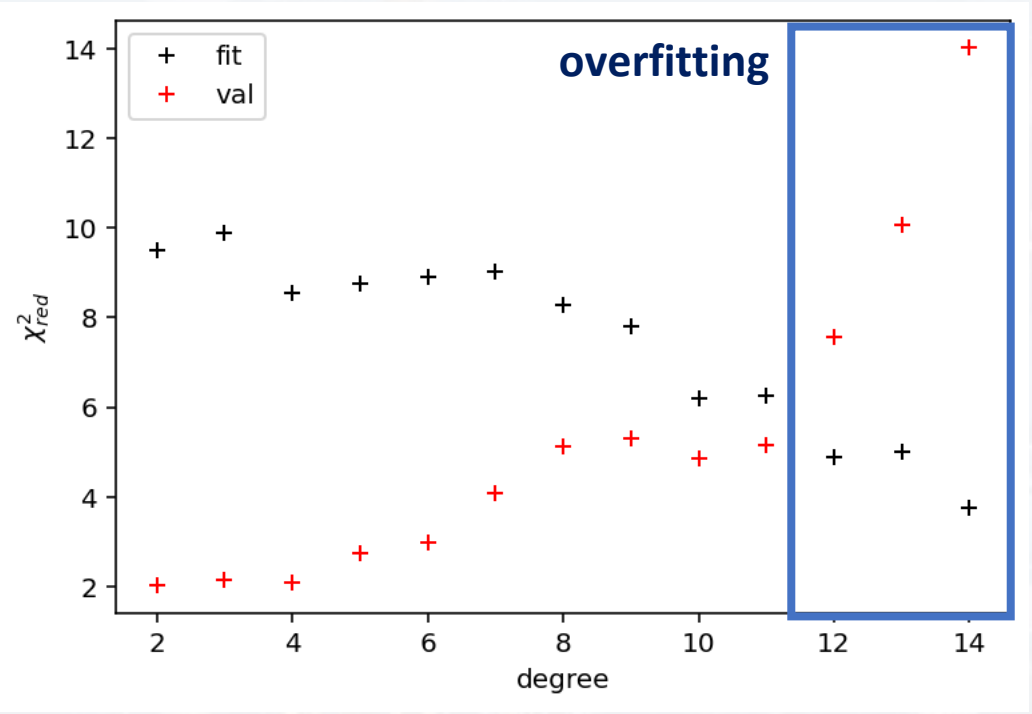


- At some point, we fit to the specific **fluctuations** of the training data!
- This is called **overfitting**!
- If therefore the fit is too specific, validation will be poor and therefore the model is actually useless!

Avoiding Overfitting and Regularization (L1 & L2)

…well, let's try and see what happens:

- At some point, we fit to the specific **fluctuations** of the training data!
- This is called **overfitting**!
- If therefore the fit is too specific, validation will be poor and therefore the model is actually useless!



Avoiding Overfitting and Regularization (L1 & L2)

Signs of **overfitting:**

- fit quality metrics (χ^2_{red}, MSE, accuracy, loss etc) are much better for the **training data** than for the **test data** set (poor generalization)

- implausible quality metrics: $\chi^2_{red} \ll 1$

  accuracy better than expected by cluster separation

How to reduce probability for **overfitting:**

- number of model parameter << number of data points

- favor the simplest model: F – test, or better: **Bayesian Model Selection** (Occam's Razor)

- run dimensionality reduction like PCA first

- k-fold cross validation

- apply regularization (see next!)

Avoiding Overfitting and Regularization (L1 & L2)





Outline

- Overfitting
- **L1 and L2 regularization**
- A Python Example
- Lagrangian Multipliers

Avoiding Overfitting and Regularization (L1 & L2)

We want to find an extreme of an **objective function**:

- minimizing: $\chi^2 = \sum_k \frac{(\hat{y}_k - y_k)^2}{\sigma_k^2}$ curve fitting

- maximizing: accuracy

$\|Y - X\beta\|^2$ linear regression

$S = -\sum_i p_i \ln p_i$ classification

$KL(p||q) = -\int p(x) \log\left[\frac{\boldsymbol{q(x)}}{\boldsymbol{p(x)}}\right] dx$ generation/ encoding

Avoiding Overfitting and Regularization (L1 & L2)

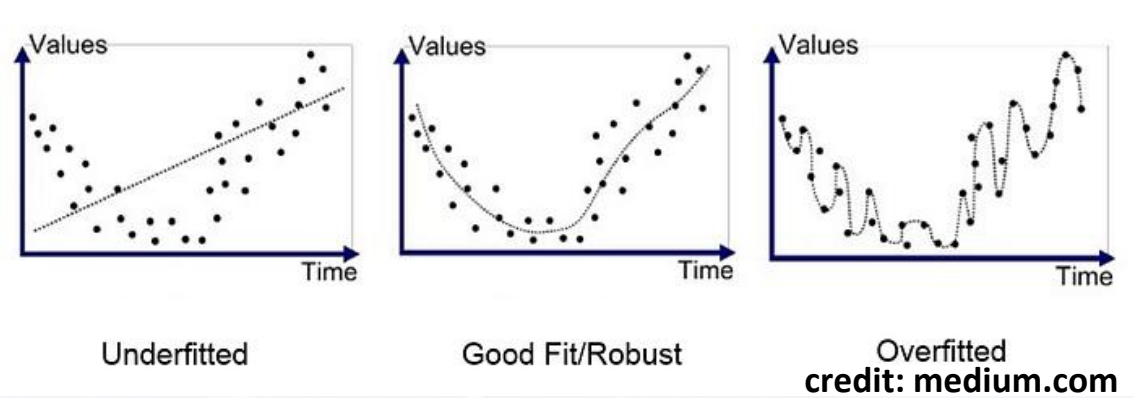
We want to find an extreme of an **objective function**:

problem: - algorithm might find an extreme, but for unreasonable values

- volume, mass, temperature (K) etc can only be positive

- values are extreme

- overfitting



credit: medium.com

- conservation:

$$S = -\sum_i p_i \ln p_i \qquad 1 = \sum_i p_i$$

Avoiding Overfitting and Regularization (L1 & L2)

λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2\}$$ linear model (see last Module)

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \boldsymbol{\lambda}\|\boldsymbol{\beta}\|^{\mathbf{1}}\}$$

the Loss Function $\boldsymbol{L(X, Y, \lambda)}$

L1 or **L**east **a**bsolute **s**hrinkage and **s**election **o**perator
- encourages **sparsity** of β
- reduces **overfitting**

hyper parameter $\boldsymbol{\lambda > 0}$

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \boldsymbol{\lambda}\|\boldsymbol{\beta}\|^{\mathbf{2}}\}$$

L2 or **Ridge**
- **penalizes large $\boldsymbol{\beta}$**

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \boldsymbol{\lambda}\, \boldsymbol{max(0, -\beta)}\}$$

- **penalizes negative $\boldsymbol{\beta}$**

…and so on

Avoiding Overfitting and Regularization (L1 & L2)

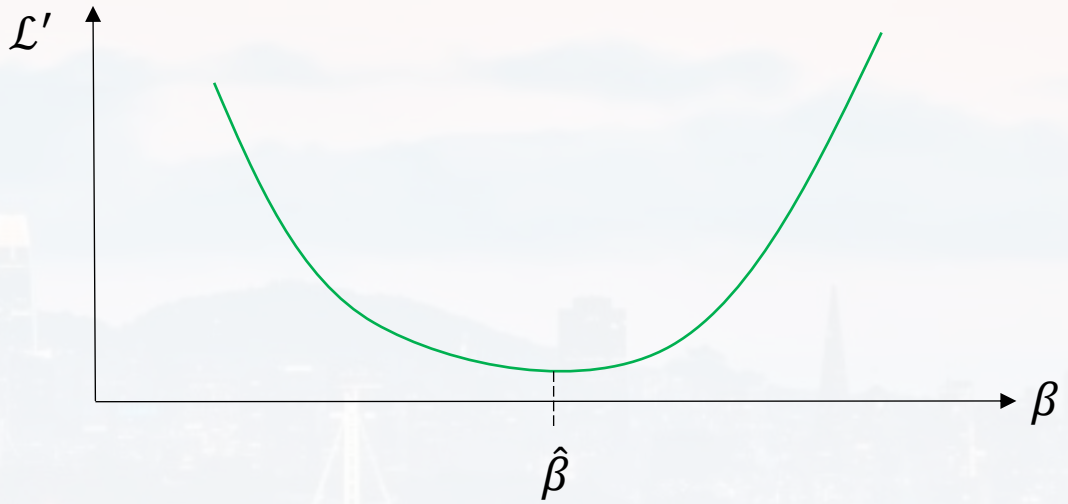
λ *Lagrangian Multiplier*

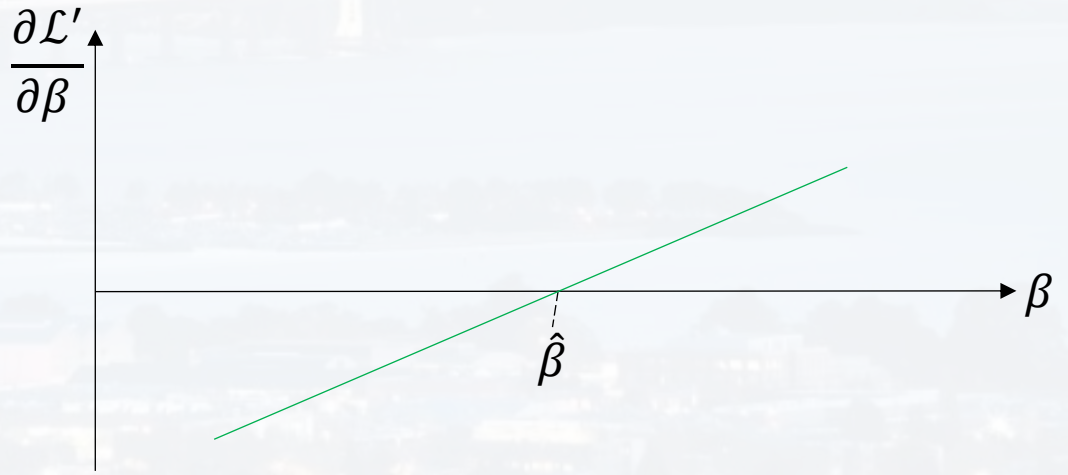
We want to find an extreme of an **objective function**:

$$\hat{\beta} = \underset{\beta}{argmin} \{\|Y - X\beta\|^2 + \boldsymbol{\lambda_1}\|\boldsymbol{\beta}\|^{\mathbf{1}} + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\mathbf{2}}\}$$

the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

$$\mathcal{L}' = \|Y - X\beta\|^2$$





Avoiding Overfitting and Regularization (L1 & L2)

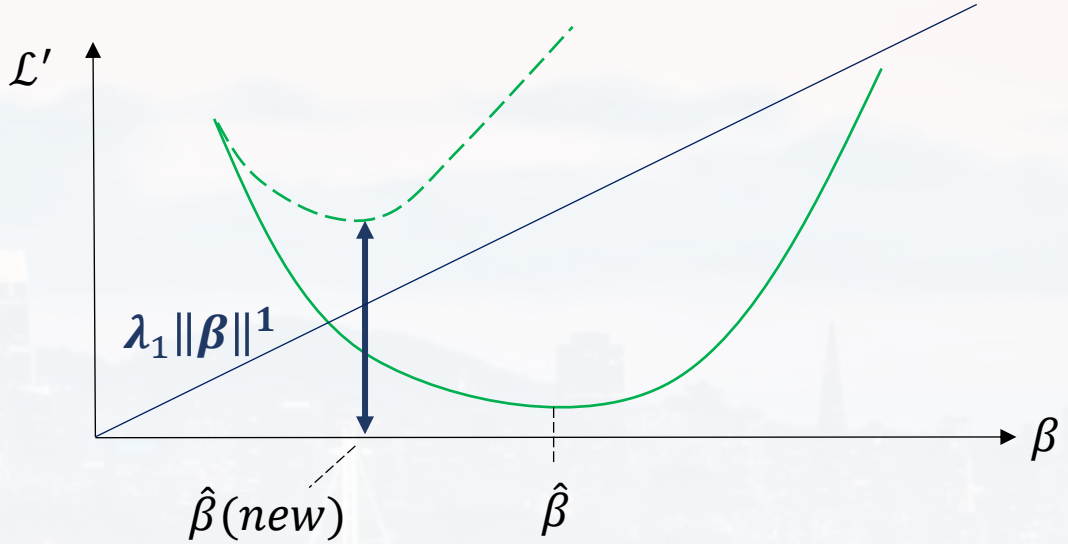
λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

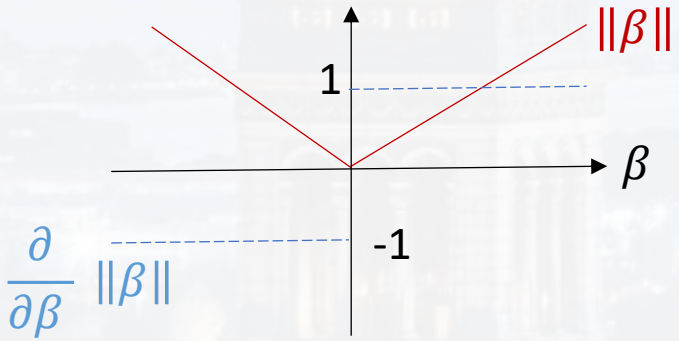
$$\hat{\beta} = \underset{\beta}{argmin}\left\{\|Y - X\beta\|^2 + \boldsymbol{\lambda_1}\|\boldsymbol{\beta}\|^{\boldsymbol{1}} + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\boldsymbol{2}}\right\}$$

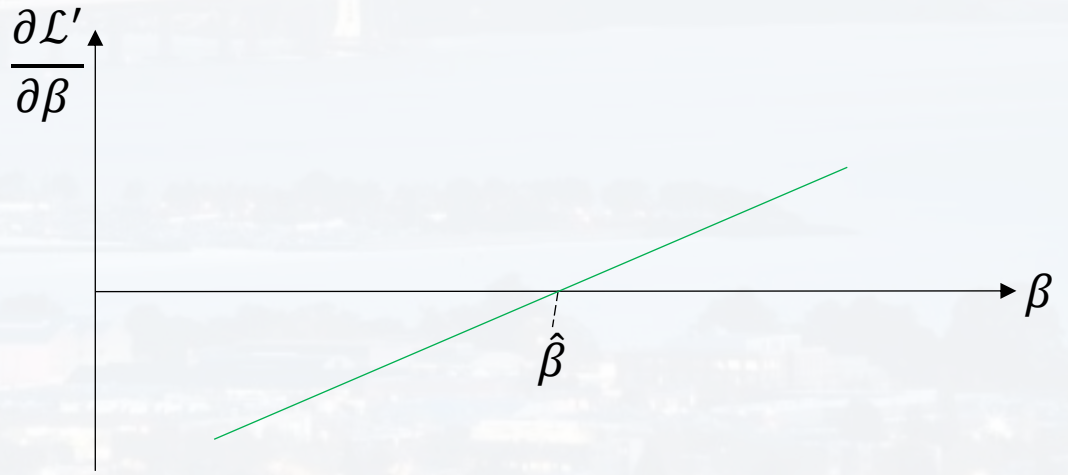
the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

$$\mathcal{L}' = \|Y - X\beta\|^2$$



$$\frac{\partial}{\partial\beta}\left[\|Y - X\beta\|^2 + \boldsymbol{\lambda_1}\|\boldsymbol{\beta}\|^{\boldsymbol{1}}\right] = \frac{\partial\mathcal{L}'}{\partial\beta} + \boldsymbol{\lambda_1}\frac{\partial}{\partial\beta}\|\boldsymbol{\beta}\|^{\boldsymbol{1}}$$





Avoiding Overfitting and Regularization (L1 & L2)

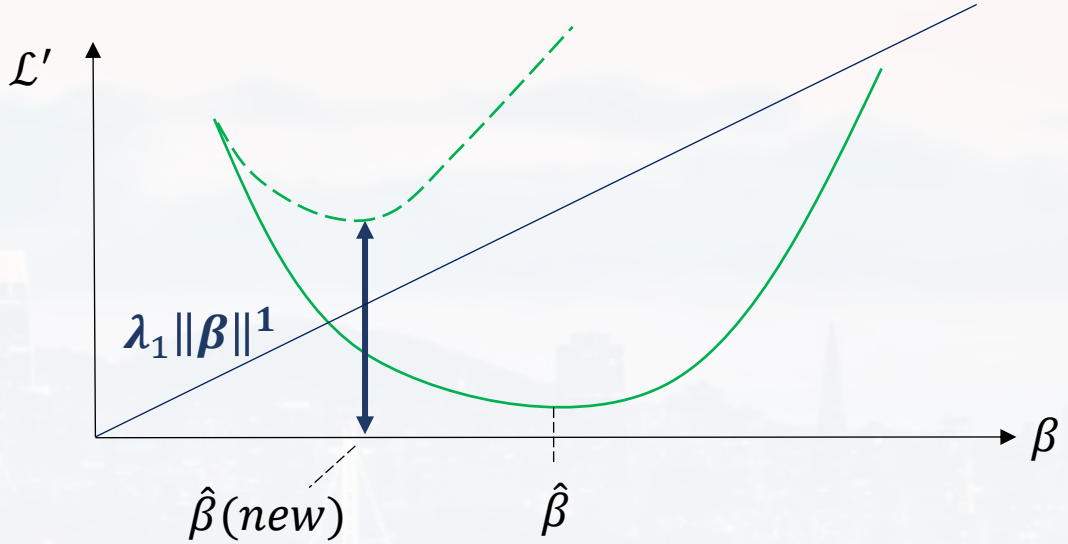
λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \lambda_1\|\boldsymbol{\beta}\|^1 + \lambda_2\|\boldsymbol{\beta}\|^2\}$$

the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

$$\mathcal{L}' = \|Y - X\beta\|^2$$

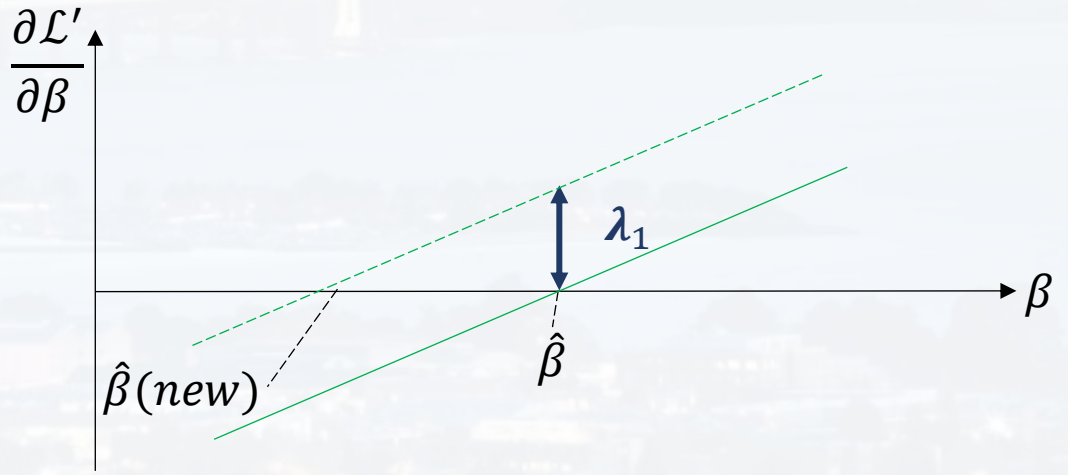


$$\frac{\partial}{\partial\beta}\left[\|Y - X\beta\|^2 + \lambda_1\|\boldsymbol{\beta}\|^1\right] = \frac{\partial\mathcal{L}'}{\partial\beta} + \lambda_1 sign(\boldsymbol{\beta})$$

$$= \frac{\partial\mathcal{L}'}{\partial\beta} \mp \lambda_1$$

shifts ***if*** there is a $\boldsymbol{\beta}$, but not sensitive to its magnitude

→ large λ_1 encourages sparsity & prevents over fitting!



Avoiding Overfitting and Regularization (L1 & L2)

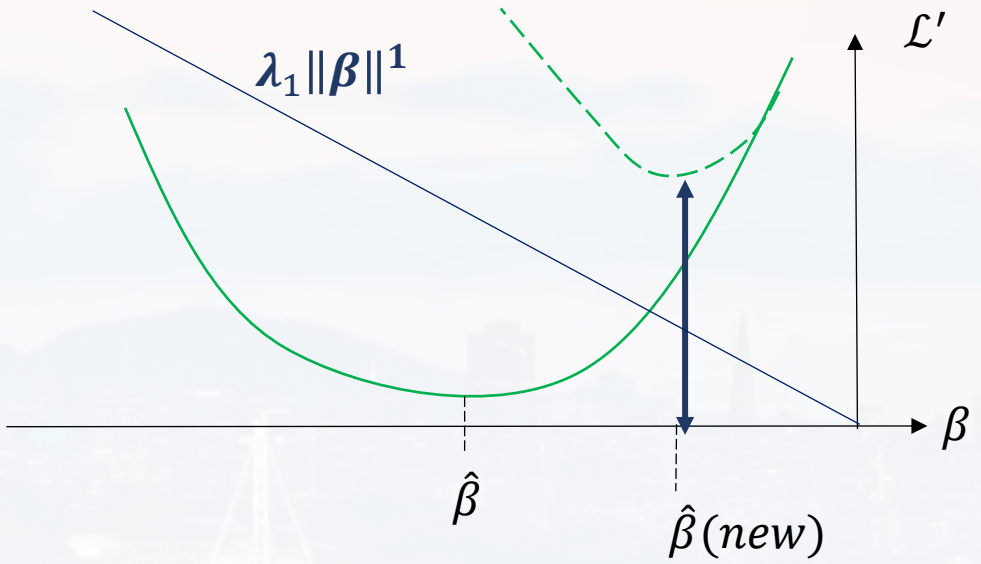
λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \lambda_1\|\boldsymbol{\beta}\|^1 + \lambda_2\|\boldsymbol{\beta}\|^2\}$$

the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

$$\mathcal{L}' = \|Y - X\beta\|^2$$

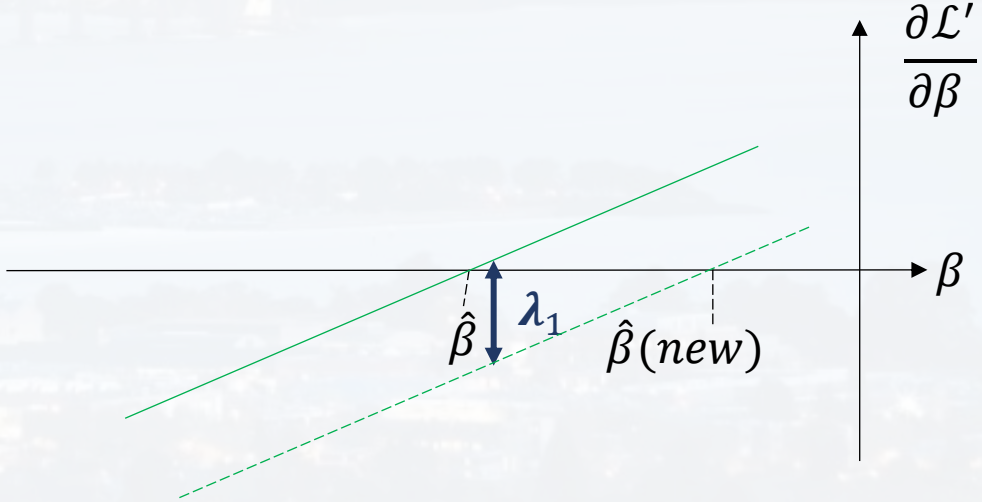


$$\frac{\partial}{\partial\beta}\left[\|Y - X\beta\|^2 + \lambda_1\|\boldsymbol{\beta}\|^1\right] = \frac{\partial\mathcal{L}'}{\partial\beta} + \lambda_1 sign(\boldsymbol{\beta})$$

$$= \frac{\partial\mathcal{L}'}{\partial\beta} \mp \lambda_1$$

shifts ***if*** there is a $\boldsymbol{\beta}$, but not sensitive to its magnitude

→ large λ_1 encourages sparsity & prevents over fitting!



Avoiding Overfitting and Regularization (L1 & L2)

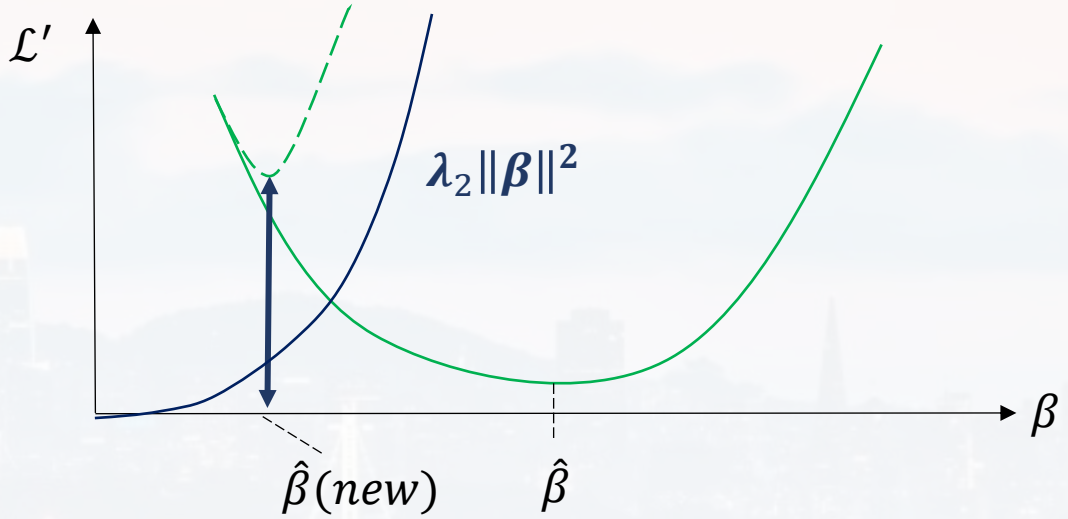
λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

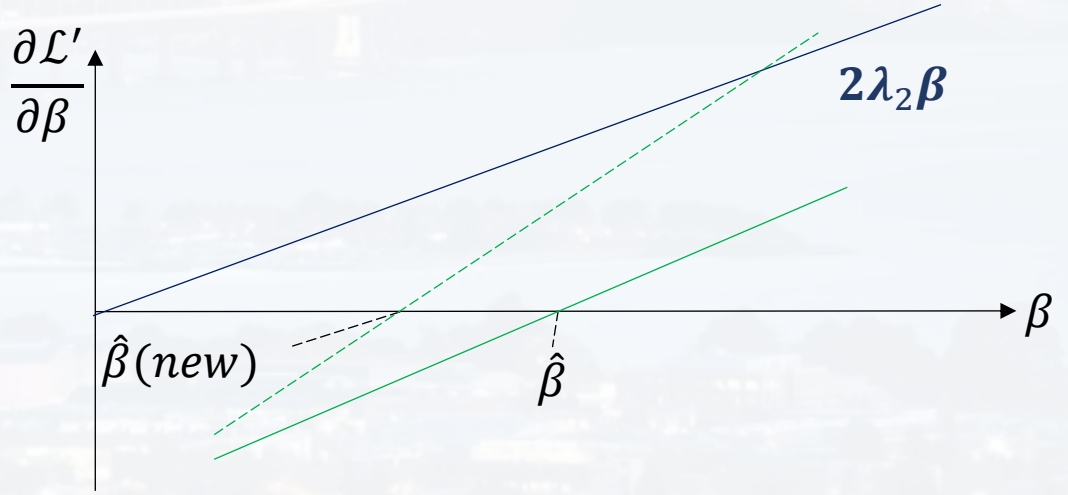
$$\hat{\beta} = \underset{\beta}{argmin} \{\|Y - X\beta\|^2 + \boldsymbol{\lambda_1}\|\boldsymbol{\beta}\|^{\mathbf{1}} + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\mathbf{2}}\}$$

the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

$$\mathcal{L}' = \|Y - X\beta\|^2$$



$$\frac{\partial}{\partial\beta}\left[\|Y - X\beta\|^2 + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\mathbf{2}}\right] = \frac{\partial\mathcal{L}'}{\partial\beta} + \mathbf{2}\boldsymbol{\lambda_2}\boldsymbol{\beta}$$



shifts according to the **magnitude** of $\boldsymbol{\beta}$,

→ large $\boldsymbol{\lambda_2}$ encourages smaller magnitudes for $\boldsymbol{\beta}$

Avoiding Overfitting and Regularization (L1 & L2)

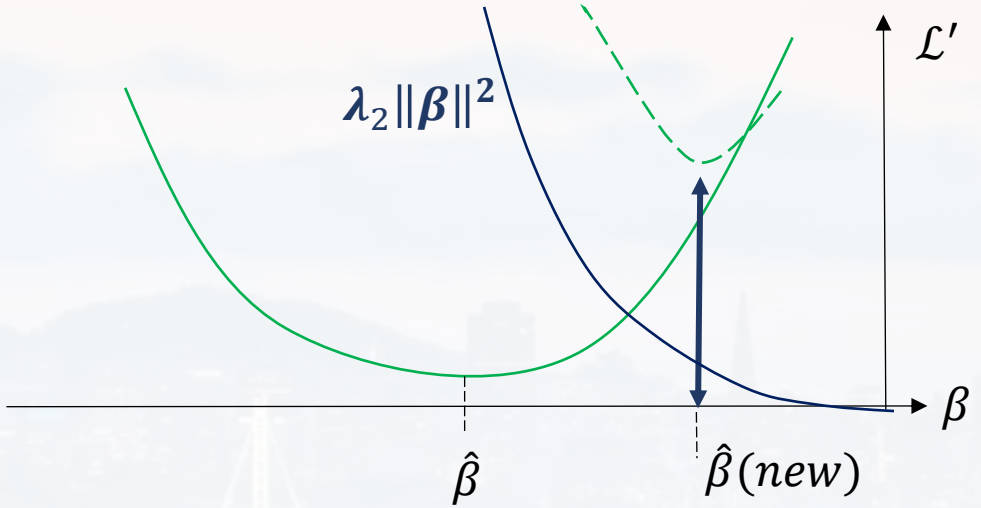
λ *Lagrangian Multiplier*

We want to find an extreme of an **objective function**:

$$\hat{\beta} = \underset{\beta}{argmin}\{\|Y - X\beta\|^2 + \boldsymbol{\lambda_1}\|\boldsymbol{\beta}\|^{\mathbf{1}} + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\mathbf{2}}\}$$

the Loss Function
$\boldsymbol{L(X, Y, \lambda)}$

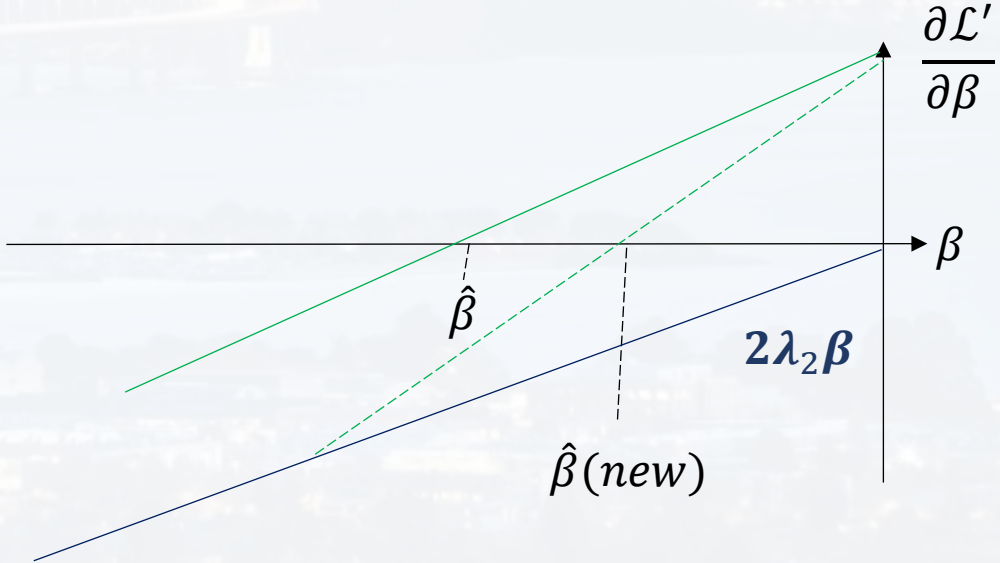
$$\mathcal{L}' = \|Y - X\beta\|^2$$



$$\frac{\partial}{\partial\beta}\left[\|Y - X\beta\|^2 + \boldsymbol{\lambda_2}\|\boldsymbol{\beta}\|^{\mathbf{2}}\right] = \frac{\partial\mathcal{L}'}{\partial\beta} + \mathbf{2}\boldsymbol{\lambda_2}\boldsymbol{\beta}$$

shifts according to the **magnitude** of $\boldsymbol{\beta}$,

→ large $\boldsymbol{\lambda_2}$ encourages smaller magnitudes for $\boldsymbol{\beta}$



Avoiding Overfitting and Regularization (L1 & L2)

note:

- how to implement L1 and L2 regularization for lin. regression in Python depends on the library (see documentation)

- "Elastic Net" balances L1 and L2: $\alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$

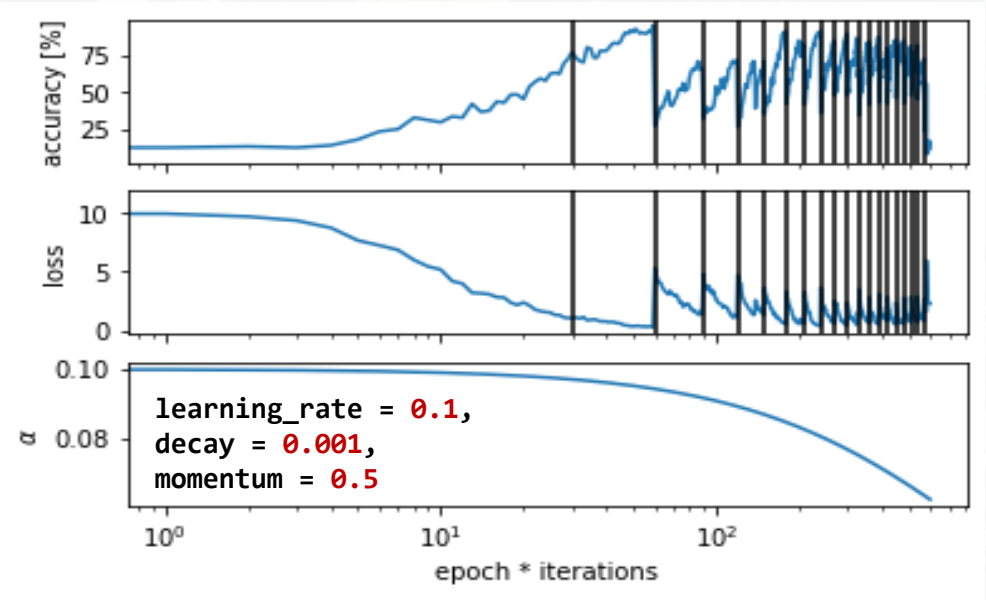
$$\lambda_1 = \alpha_1 \alpha_2$$

$$\lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$$

- L1: deals with highly correlated regressors (sparicty)

- L2: deals with large facors (keeps solution stable)

- regularisation is pretty common, not only for lin. models



LeNet numpy only

```
W = np.load('weightsC1.npy')
W[:,:,0]
```

	0	1	2	3	4	5
0	-15923.3	-16647.4	-16277.1	-16993.1	-15715.7	-15390.1
1	9795.1	8468.91	9110.83	8205.6	9852.06	11061
2	37956.4	36572.6	37324.2	37008.4	37485.4	38515.2
3	39686.6	39131	39087.4	39614.6	39421.7	39876.7
4	25465.3	26270.1	25232.4	25836.7	25590.5	25270.9

Avoiding Overfitting and Regularization (L1 & L2)

note:

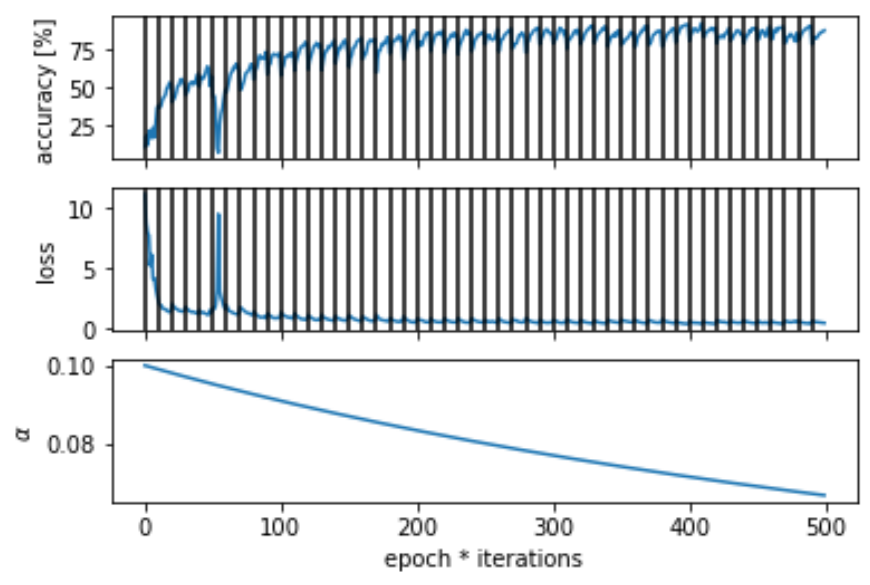
- how to implement L1 and L2 regularization for lin. regression in Python depends on the library (see documentation)

- "Elastic Net" balances L1 and L2: $\alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$

- L1: deals with highly correlated regressors (sparicty)

- L2: deals with large facors (keeps solution stable)

- regularisation is pretty common, not only for lin. models



LeNet numpy only

L1 and L2 regualrization

Avoiding Overfitting and Regularization (L1 & L2)





Outline

- Overfitting
- L1 and L2 regularization
- **A Python Example**
- Lagrangian Multipliers

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

'Diabetes2.csv'

X

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	label
0	6	148.0	72.0	35.00	184.42	33.6	0.63	50	diabetes
1	1	85.0	66.0	29.00	75.91	26.6	0.35	31	healthy
2	8	183.0	64.0	31.34	155.87	23.3	0.67	32	diabetes
3	1	89.0	66.0	23.00	94.00	28.1	0.17	21	healthy
4	0	137.0	40.0	35.00	168.00	43.1	2.29	33	diabetes

Y

WalkThroughL1L2.ipynb

1) running standard logistic model
2) exploring L1 and L2
3) the sparse model

for robust results: split 60/40

```
location = FindMyFile('Diabetes2.csv')
Data     = pd.read_csv(location, sep = ';')

X        = Data.drop(['label'], axis = 1)
Y        = Data['label']

X_Train, X_Test, Y_Train, Y_Test = train_test_split(X, Y, test_size = 0.40,
                                                    random_state = 1)


scaler   = StandardScaler()
XS_Train = scaler.fit_transform(X_Train)
XS_Test  = scaler.transform(X_Test)
```

Avoiding Overfitting and Regularization (L1 & L2)


```
scaler   = StandardScaler()
XS_Train = scaler.fit_transform(X_Train)
XS_Test  = scaler.transform(X_Test)
```

adding intercept

WalkThroughL1L2.ipynb

1) **running standard logistic model**
2) exploring L1 and L2
3) the sparse model

```
XS_Train_add = sm.add_constant(pd.DataFrame(XS_Train, columns = X.columns))
Ydumm        = pd.get_dummies(Y_Train).reset_index(drop = True)

my_model     = sm.GLM(Ydumm, XS_Train_add, family = sm.families.Binomial())
result       = my_model.fit()
result.summary()
```

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

some coefficients seem to be not significant

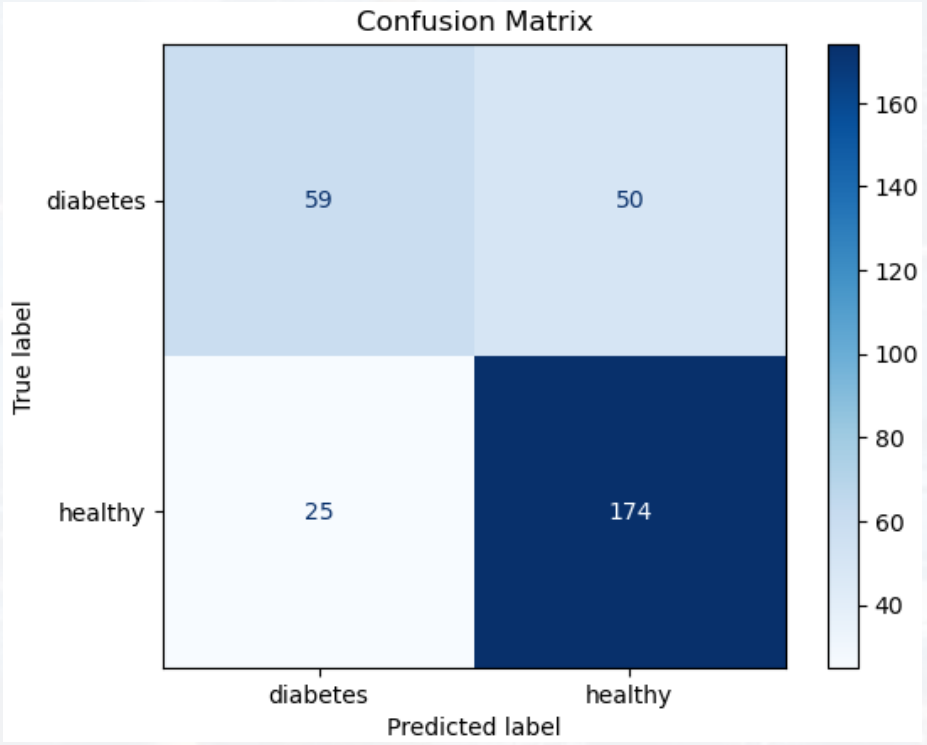
Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

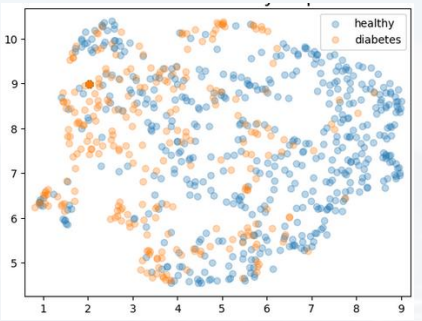
`WalkThroughL1L2.ipynb`

1) **running standard logistic model**
2) exploring L1 and L2
3) the sparse model

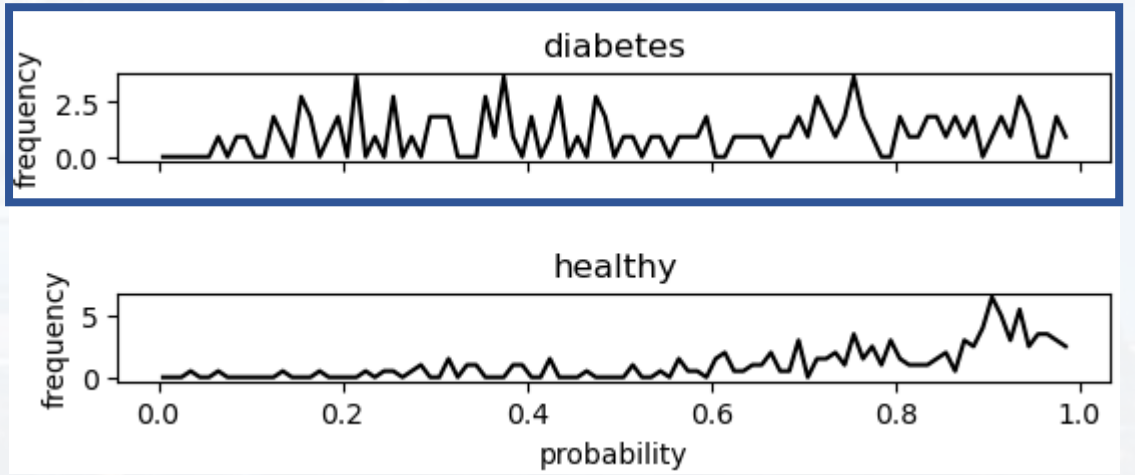
```
my_model     = sm.GLM(Ydumm, XS_Train_add, family = sm.families.Binomial())
result       = my_model.fit()
result.summary()
```




```
Accuracy is  75.65%
```



model is not very reliable for class “diabetes”



Avoiding Overfitting and Regularization (L1 & L2)

"elastic net"

$$\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right) \qquad \mathcal{L}' = \|Y - X\beta\|^2$$

$$\lambda_1 = \alpha_1 \alpha_2 \qquad \lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$$

$\alpha_2 = 1$: L1 regularization
$\alpha_2 = 0$: L2 regularization

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) **exploring L1 and L2**
3) the sparse model

Let's try some values for α_1 and α_2:

```
my_model      = sm.GLM(Ydumm, XS_Train_add, family = sm.families.Binomial())

alpha1 = 0.1
alpha2 = 0.1

result_reg = my_model.fit_regularized(method = 'elastic_net',
                                      alpha  = alpha1,
                                      L1_wt  = alpha2)
```

```
Accuracy is  75.65%
```

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

"elastic net" $\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$

$\mathcal{L}' = \|Y - X\beta\|^2$

`WalkThroughL1L2.ipynb`

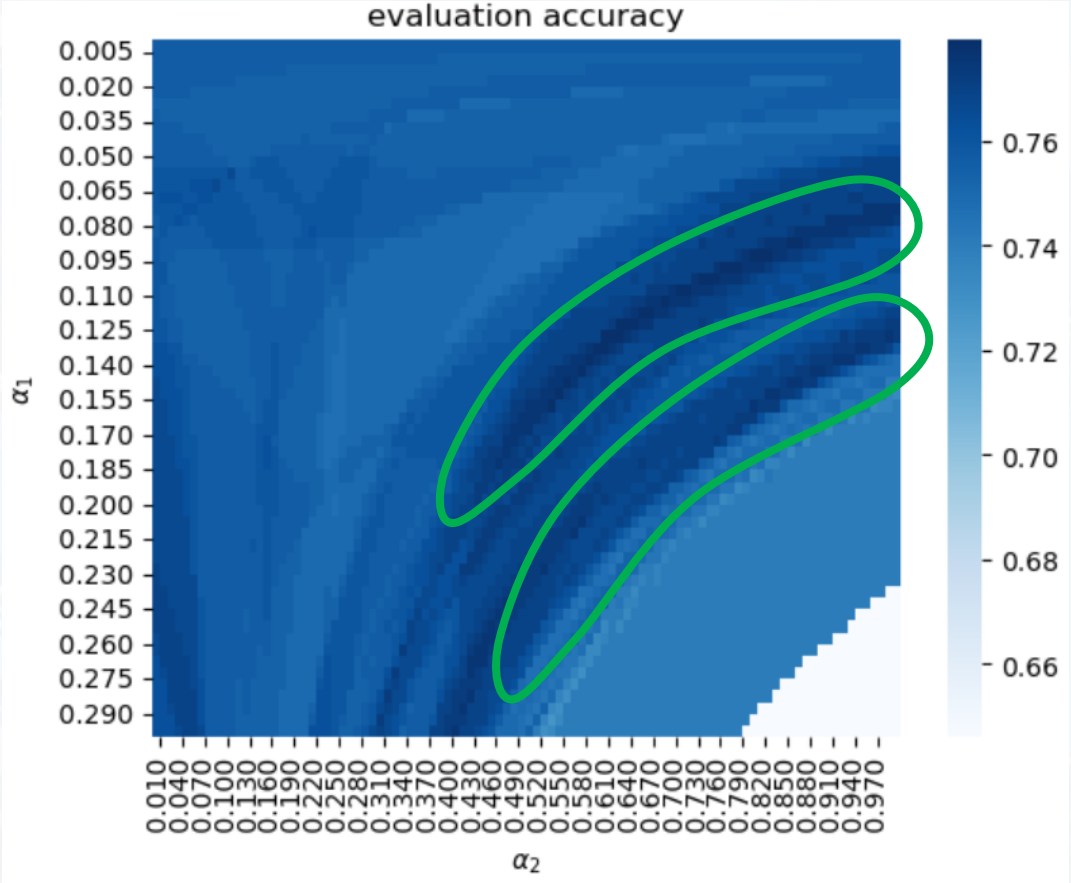
1) running standard logistic model
2) **exploring L1 and L2**
3) the sparse model

$\lambda_1 = \alpha_1 \alpha_2 \quad \lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$

$\alpha_2 = 1$: L1 regularization
$\alpha_2 = 0$: L2 regularization

scanning for α_1 and α_2:

```
best accuracy:  77.92%
alpha_1 = 0.085
alpha_2 = 0.85
```



Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

"elastic net" $\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$

$\mathcal{L}' = \|Y - X\beta\|^2$

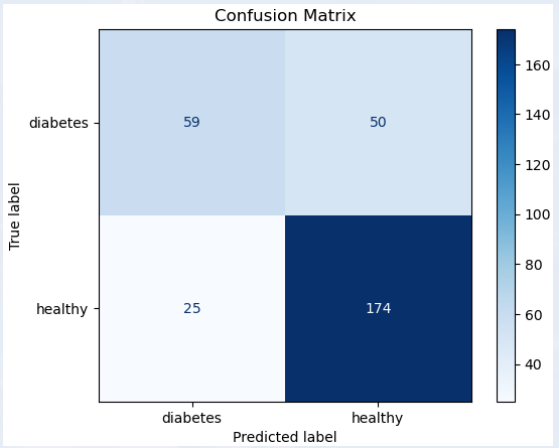
$\lambda_1 = \alpha_1 \alpha_2 \quad \lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$

$\alpha_2 = 1$: L1 regularization
$\alpha_2 = 0$: L2 regularization

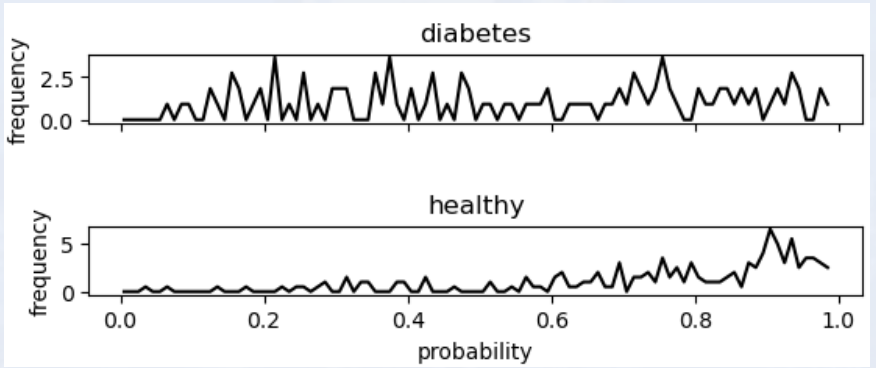
`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) **exploring L1 and L2**
3) the sparse model

no regularization

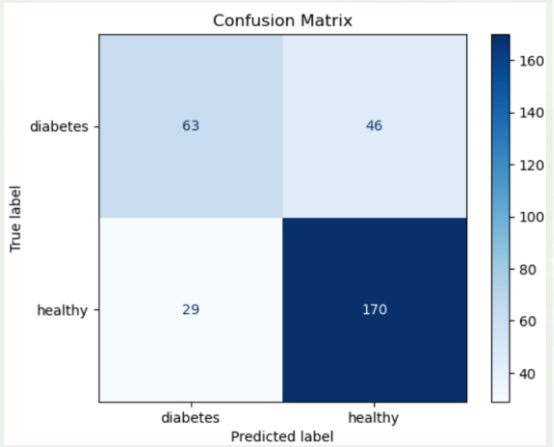


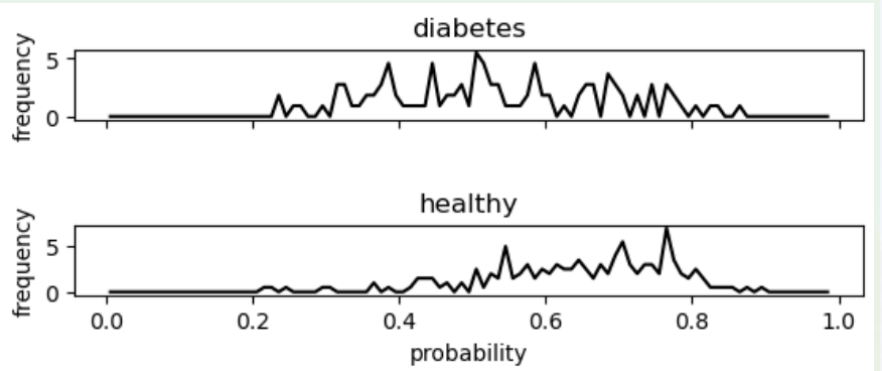

```
Accuracy is  75.65%
```



L1 & L2 regularization

```
best accuracy:  77.92%
alpha_1 = 0.085
alpha_2 = 0.85
```





Avoiding Overfitting and Regularization (L1 & L2)

"elastic net" $\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$ $\mathcal{L}' = \|Y - X\beta\|^2$

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) **exploring L1 and L2**
3) the sparse model

coefficients and confidence intervals:

- `result_reg` only returns coefficient values, but no confidence intervals

- solution: bootstrapping (see Module 3)

```
lower, upper = boot_strap_coeff(np.array(XS_Train_add),
                                Ydumm, best_a1    = BestAlpha1,
                                       best_a2    = BestAlpha1,
                                       Nboot      = 1000)
```

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

"elastic net" $\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right)$ $\mathcal{L}' = \|Y - X\beta\|^2$

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) **exploring L1 and L2**
3) the sparse model

no regularization

	Coef.	[0.025	0.975]
const	-0.9067	-1.1574	-0.6559
Pregnancies	0.2391	-0.0233	0.5015
Glucose	1.0384	0.7406	1.3362
BloodPressure	-0.2400	-0.5122	0.0322
SkinThickness	-0.0690	-0.3650	0.2269
Insulin	0.0839	-0.1994	0.3673
BMI	0.8625	0.5380	1.1870
DiabetesPedigreeFunction	0.1161	-0.1303	0.3626
Age	0.3368	0.0535	0.6201

L1 & L2 regularization

some coefficients are consistent with zero → sparsity (L1)

magnitudes of coefficients are smaller (L2)

conf 0.025	best	conf 0.975
-0.644043	-0.339928	-0.378195
0.000000	0.000000	0.276625
0.468804	0.598115	0.723600
-0.132069	0.000000	0.074831
0.000000	0.000000	0.207292
0.000000	0.000000	0.356646
0.287730	0.282465	0.525576
-0.034453	0.000000	0.235977
0.031497	0.000703	0.296114

Avoiding Overfitting and Regularization (L1 & L2)

some coefficients are consistent with zero → sparsity (L1)

→ a simpler model (fewer coefficients, **dimensionality reduction**) would perform as well as the "complete" model

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

```
XS_Test_sparse  = XS_Test[['Glucose', 'BloodPressure', 'BMI',
                           'DiabetesPedigreeFunction', 'Age']]

XS_Train_sparse = XS_Train[['const', 'Glucose', 'BloodPressure', 'BMI',
                            'DiabetesPedigreeFunction', 'Age']]
```

	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

no regularization – complete model

	Coef.	Std.Err.	z	P>|z|	[0.025	0.975]
const	-0.9067	0.1279	-7.0873	0.0000	-1.1574	-0.6559
Pregnancies	0.2391	0.1339	1.7857	0.0742	-0.0233	0.5015
Glucose	1.0384	0.1519	6.8343	0.0000	0.7406	1.3362
BloodPressure	-0.2400	0.1389	-1.7278	0.0840	-0.5122	0.0322
SkinThickness	-0.0690	0.1510	-0.4571	0.6476	-0.3650	0.2269
Insulin	0.0839	0.1446	0.5806	0.5615	-0.1994	0.3673
BMI	0.8625	0.1655	5.2101	0.0000	0.5380	1.1870
DiabetesPedigreeFunction	0.1161	0.1257	0.9234	0.3558	-0.1303	0.3626
Age	0.3368	0.1445	2.3300	0.0198	0.0535	0.6201

no regularization – sparse model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8985	0.127	-7.081	0.000	-1.147	-0.650
Glucose	1.0654	0.137	7.756	0.000	0.796	1.335
BloodPressure	-0.2246	0.138	-1.630	0.103	-0.495	0.045
BMI	0.8274	0.139	5.946	0.000	0.555	1.100
DiabetesPedigreeFunction	0.1151	0.124	0.932	0.351	-0.127	0.357
Age	0.4469	0.128	3.486	0.000	0.196	0.698

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

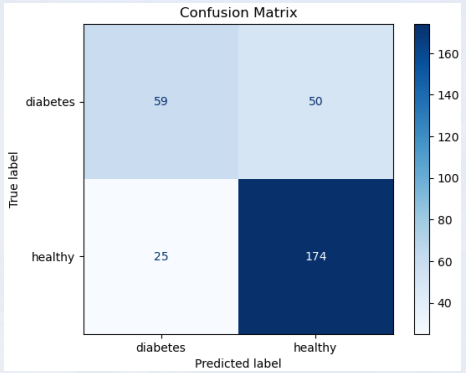
	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

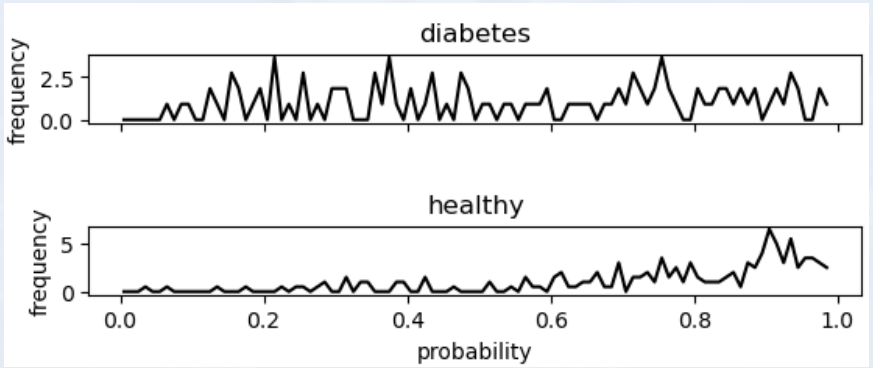
`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

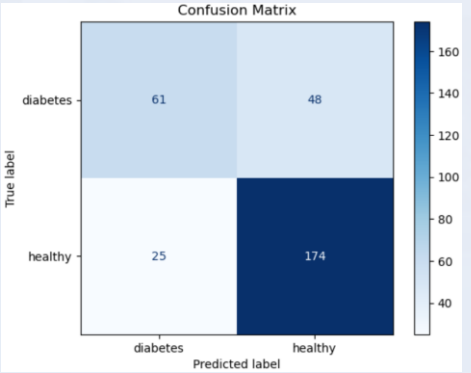
no regularization – complete model



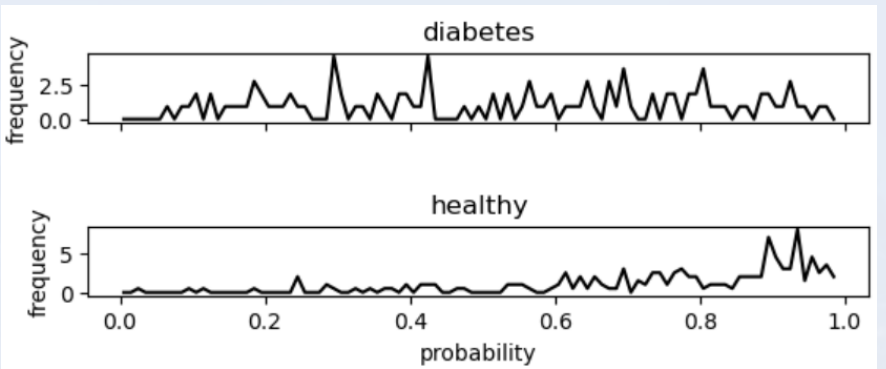
`Accuracy is 75.65%`



no regularization – sparse model



`Accuracy is 76.30%`



Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

no regularization – complete model

	Coef.	Std.Err.	z	P>|z|	[0.025	0.975]
const	-0.9067	0.1279	-7.0873	0.0000	-1.1574	-0.6559
Pregnancies	0.2391	0.1339	1.7857	0.0742	-0.0233	0.5015
Glucose	1.0384	0.1519	6.8343	0.0000	0.7406	1.3362
BloodPressure	-0.2400	0.1389	-1.7278	0.0840	-0.5122	0.0322
SkinThickness	-0.0690	0.1510	-0.4571	0.6476	-0.3650	0.2269
Insulin	0.0839	0.1446	0.5806	0.5615	-0.1994	0.3673
BMI	0.8625	0.1655	5.2101	0.0000	0.5380	1.1870
DiabetesPedigreeFunction	0.1161	0.1257	0.9234	0.3558	-0.1303	0.3626
Age	0.3368	0.1445	2.3300	0.0198	0.0535	0.6201

best regularization – sparse model

	conf 0.025	best	conf 0.975
const	-0.639295	-0.339928	-0.363785
Glucose	0.514546	0.598115	0.776440
BloodPressure	-0.120395	0.000000	0.089073
BMI	0.320461	0.282465	0.577136
DiabetesPedigreeFunction	-0.006561	0.000000	0.267181
Age	0.091881	0.000703	0.369953

Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

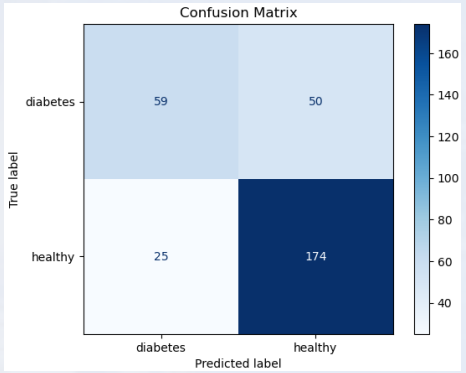
	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

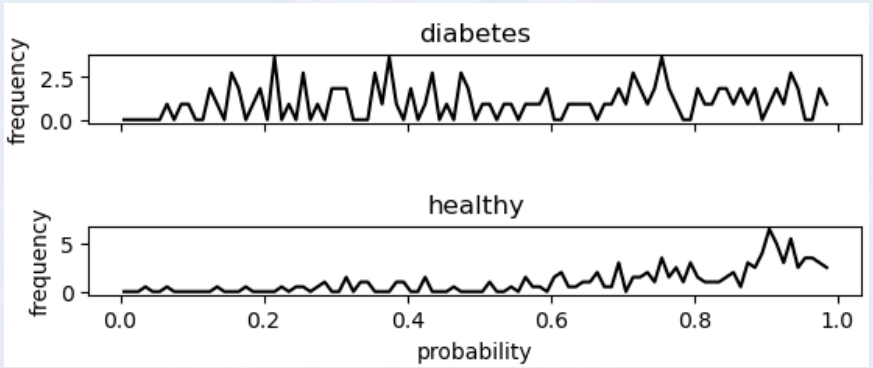
`WalkThroughL1L2.ipynb`

1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

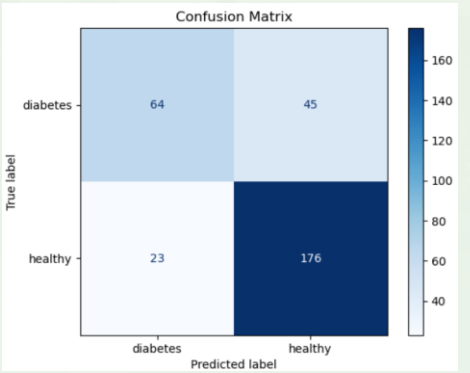
no regularization – complete model



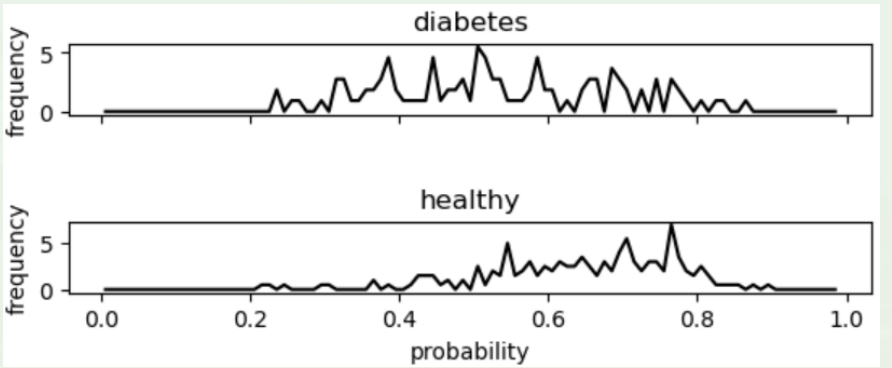

```
Accuracy is  75.65%
```



best regularization – sparse model




```
Accuracy is  77.92%
```



Avoiding Overfitting and Regularization (L1 & L2)

A Python Example

	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

"sparse" model

`WalkThroughL1L2.ipynb`

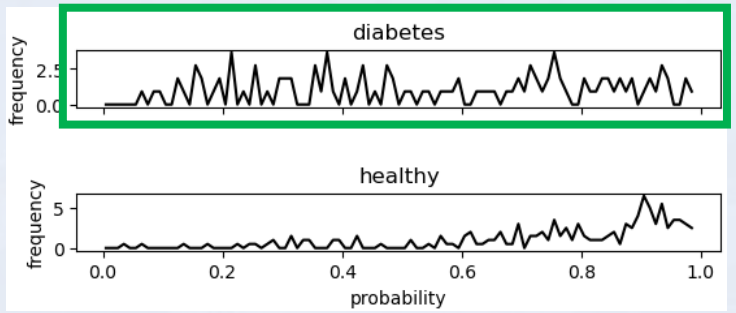
1) running standard logistic model
2) exploring L1 and L2
3) **the sparse model**

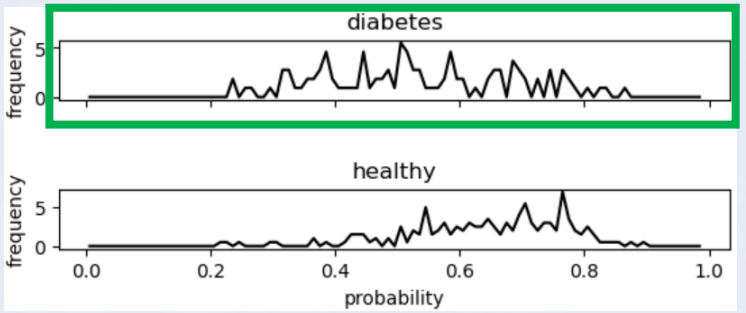
summary:

- dimensionality reduced from nine to six coefficients

- accuracy increased from 75.7% to 77.9%

- cross-entropy improved





- error ranges are slightly smaller

Avoiding Overfitting and Regularization (L1 & L2)





Outline

- Overfitting
- L1 and L2 regularization
- A Python Example
- **Lagrangian Multipliers**

Avoiding Overfitting and Regularization (L1 & L2)

More about ***Lagrangian Multiplier***:

$$\mathcal{L} = \mathcal{L}' + \alpha_1 \left(\frac{1-\alpha_2}{2} \|\boldsymbol{\beta}\|^2 + \alpha_2 \|\boldsymbol{\beta}\|^1 \right) \qquad \mathcal{L}' = \|Y - X\beta\|^2$$

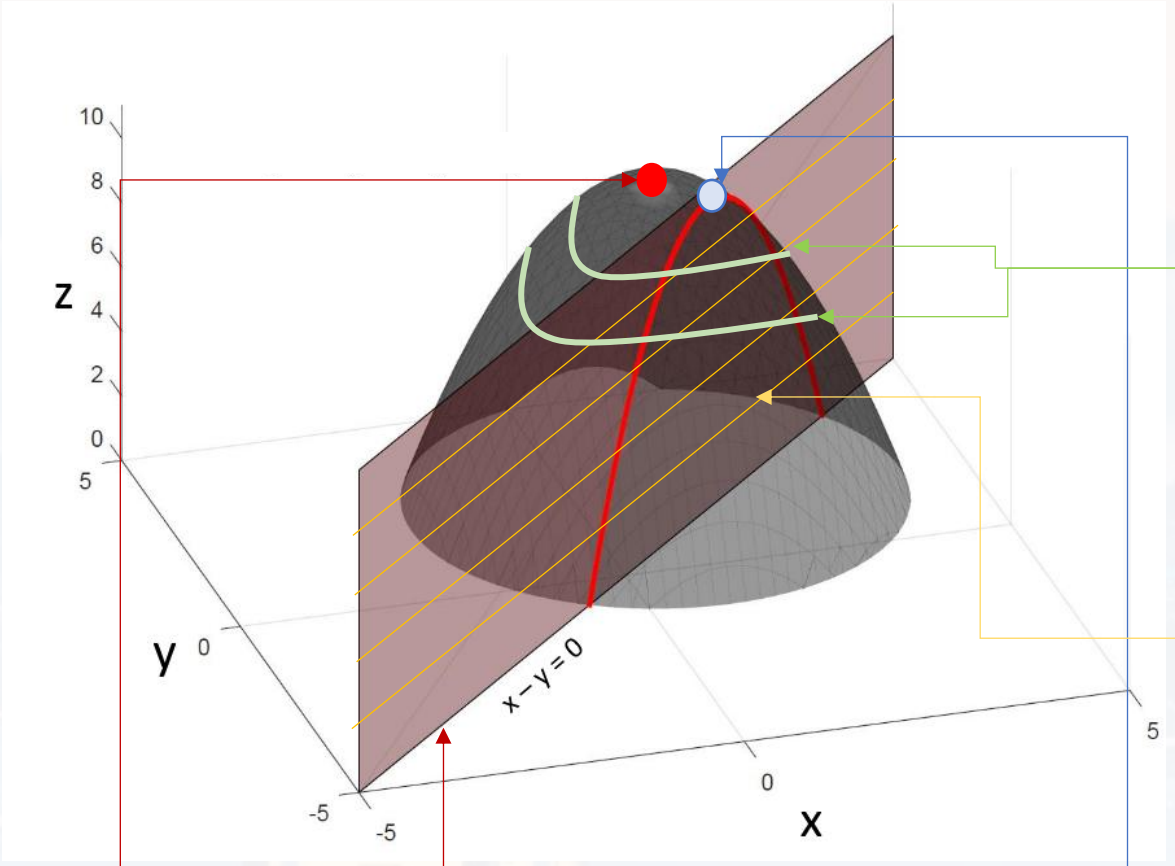
$\alpha_2 = 1$: L1 regularization
$\alpha_2 = 0$: L2 regularization

$$\lambda_1 = \alpha_1 \alpha_2 \qquad \lambda_2 = \alpha_1 \frac{1-\alpha_2}{2}$$

λ_1 and λ_2 are Langrangian Multiplier – but what does that actually mean?

Avoiding Overfitting and Regularization (L1 & L2)

More about ***Lagrangian Multiplier***:



maximum of the function

Lagrangian Multiplier
Examples

$$f(x,y) = z = -(x-2)^2 - (y-1)^2 + 10$$

level lines $f(x,y) = const$

$$df(x,y) = \frac{\partial f(x,y)}{\partial x}dx + \frac{\partial f(x,y)}{\partial y}dy = 0$$

$$= gradf\ d\vec{r} = 0$$

level lines $g(x,y) = const$

$$dg(x,y) = \frac{\partial g(x,y)}{\partial x}dx + \frac{\partial g(x,y)}{\partial y}dy = 0$$

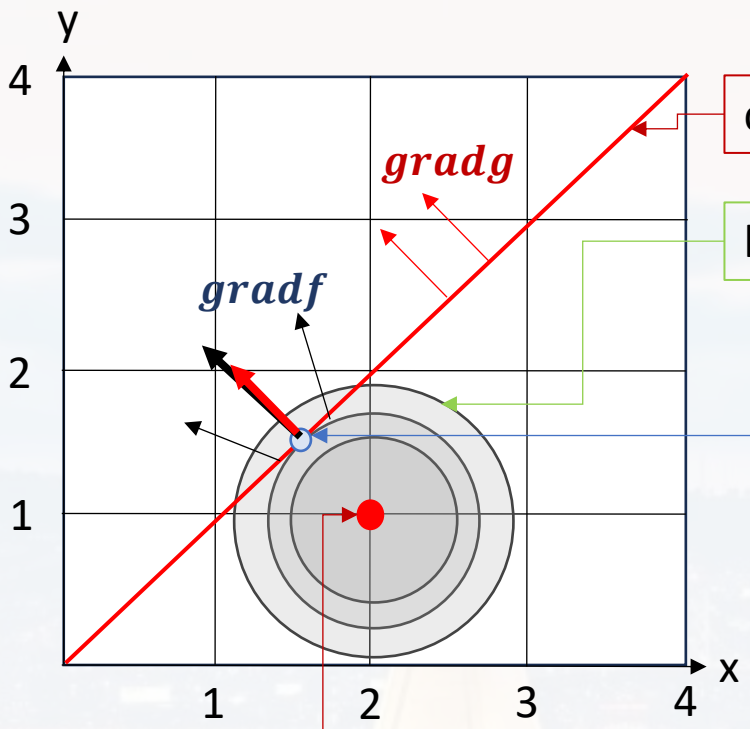
$$= gradg\ d\vec{r} = 0$$

$z = 10$ at
$x = 2$
$y = 1$

constrain $g(x,y) = x - y = 0$

maximum of the function, subject to $g(x,y)$

Avoiding Overfitting and Regularization (L1 & L2)



constrain $g(x,y) = x - y = 0$

level lines $f(x,y) = const$

$z = 10$ at
$x = 2$
$y = 1$

maximum of the function, subject to $g(x,y)$

the maximum of $f(x,y)$ subject to $g(x,y)$ located where:

$$df(x,y) = dg(x,y)$$

$$gradf\ d\vec{r} = gradg\ d\vec{r}$$

$$gradf = gradg$$

Both gradients need to point in the same direction
(hence, can be multiplied with a constant, say λ)!

$$gradf = \lambda\ gradg$$

λ **Lagrangian Multiplier**

the maximum of $f(x,y)$ subject to $g(x,y)$

Lagrangian Multiplier
Examples

$$gradf = \lambda\, gradg$$

$$f(x,y) - \lambda g(x,y) = const$$

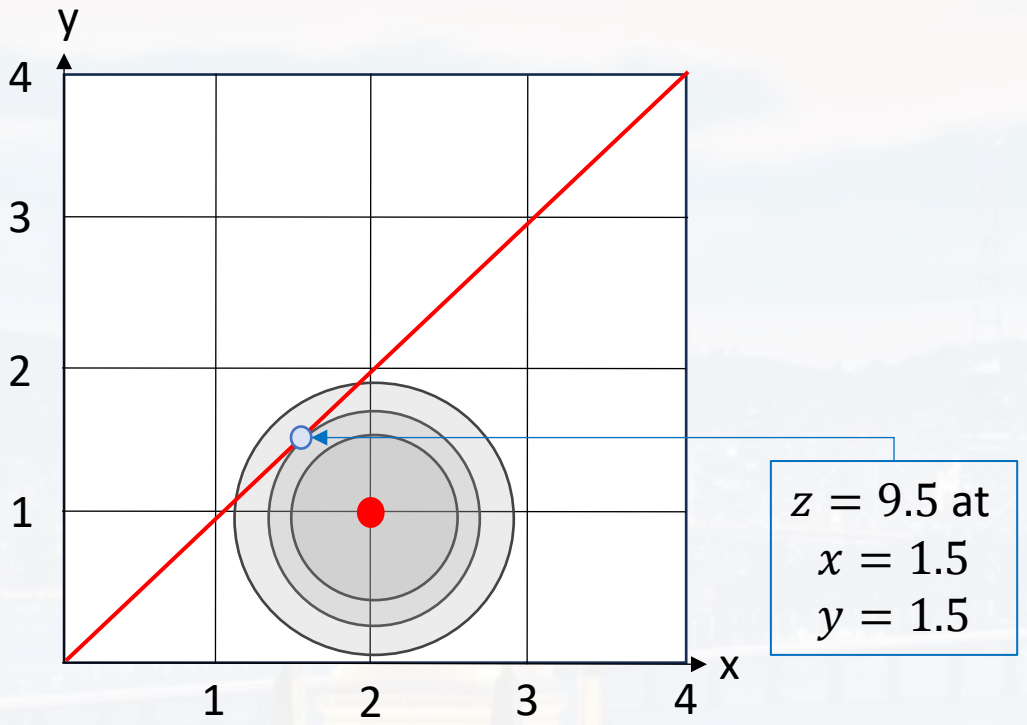
the Lagrangian
$\boldsymbol{L(x,y,\lambda)}$

more general:

$$L(x_1, x_2, \dots, x_i, x_N, \lambda_1, \lambda_2, \dots, \lambda_k, \lambda_K) = f(x_1, x_2, \dots, x_i, x_N) - \sum_{k=1}^{K} \lambda_k\, g_k(x_1, x_2, \dots, x_i, x_N)$$

note:
- ***N* dimensions and $K \leq N$ constrains**
- we need to solve *N* (from the gradient) + *K* equations by using the constrains
- optimization: more robust results (most common L1 and L2 regularization, as before)
- machine learning: including constrains in loss function (see later)

the maximum of $f(x, y)$ subject to $g(x, y)$



maximum of the function

Lagrangian Multiplier
Examples

$$f(x, y) = z = -(x-2)^2 - (y-1)^2 + 10$$

constrain $g(x, y) = x - y = 0$

constrain $x = y$

$$x = 1.5$$
$$y = 1.5$$

$$f(1.5, 1.5) = 9.5$$

$$gradf = \lambda\, gradg$$

$$\frac{\partial f(x,y)}{\partial x} = \lambda \frac{\partial g(x,y)}{\partial x} \qquad -2(x-2) = \lambda$$

$$\frac{\partial f(x,y)}{\partial y} = \lambda \frac{\partial g(x,y)}{\partial y} \qquad -2(y-1) = -\lambda$$

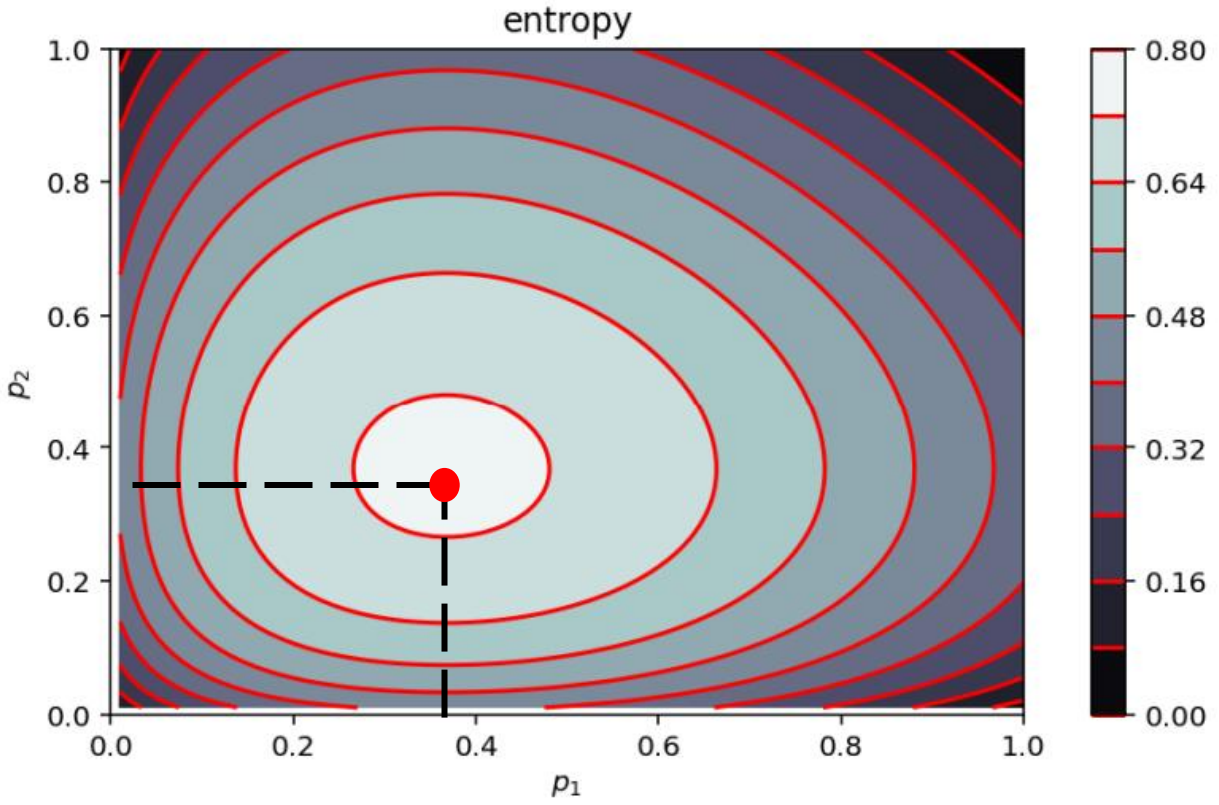
$$y = -x + 3$$

maximum entropy of flipping a coin:

$$f(p_1, p_2) = -p_1 \ln p_1 - p_2 \ln p_2$$

subject to

$$g(p_1, p_2) = p_1 + p_2 = 1$$



absolute maximum:

$$\frac{\partial f(p_1, p_2)}{\partial p_1} = 0 \qquad \frac{\partial f(p_1, p_2)}{\partial p_2} = 0$$

$$-\ln p_1 - 1 = 0 \qquad -\ln p_2 - 1 = 0$$

$$p_1 = p_2 = \frac{1}{e}$$

$$f\left(\frac{1}{e}, \frac{1}{e}\right) = \frac{2}{e} \approx 0.74$$

$$p_1 + p_2 = \frac{2}{e} > 1$$

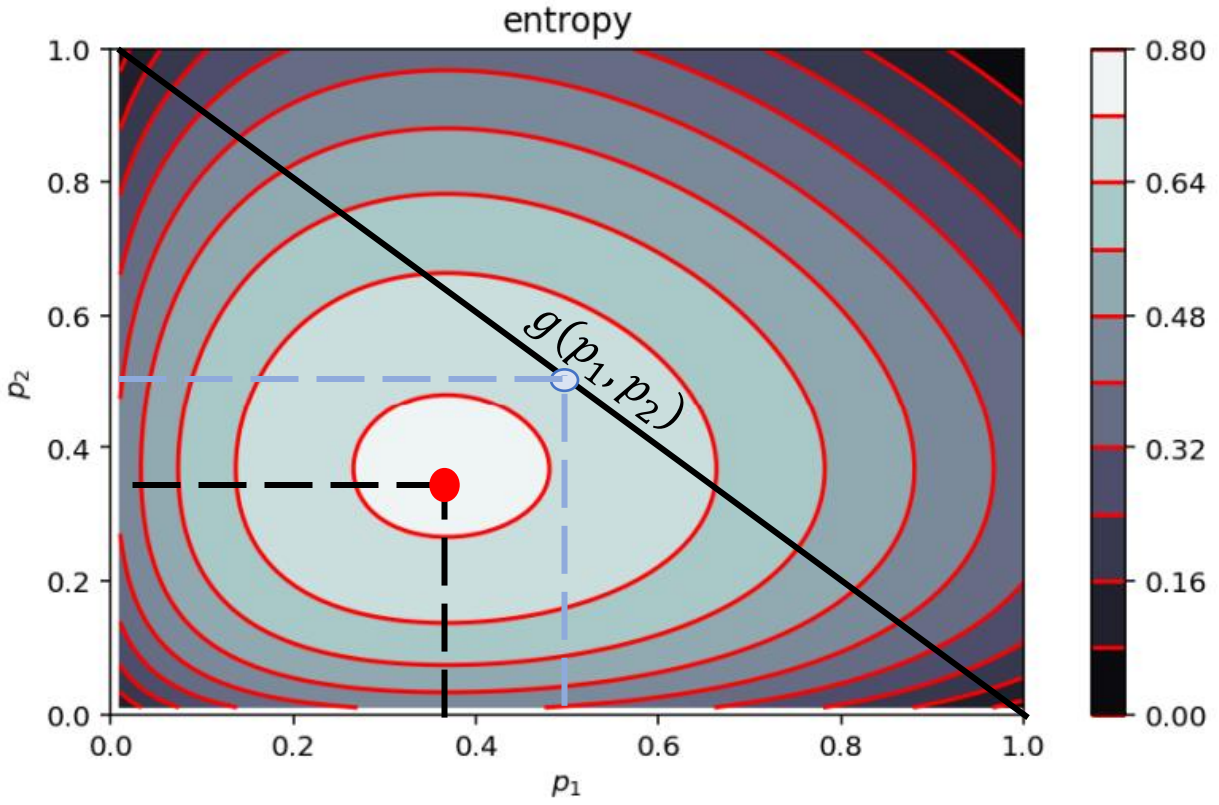
Lagrangian Multiplier
Examples

maximum entropy of flipping a coin:

$$f(p_1, p_2) = -p_1 \ln p_1 - p_2 \ln p_2$$

subject to

$$g(p_1, p_2) = p_1 + p_2 = 1$$



maximum subject to $g(p_1, p_2)$:

$$\frac{\partial f(p_1, p_2)}{\partial p_1} = \lambda \frac{\partial g(p_1, p_2)}{\partial p_1}$$

$$\frac{\partial f(p_1, p_2)}{\partial p_2} = \lambda \frac{\partial g(p_1, p_2)}{\partial p_2}$$

$$-\ln p_1 - 1 = \boldsymbol{\lambda} \qquad -\ln p_2 - 1 = \boldsymbol{\lambda}$$

$$\boldsymbol{p_1 = p_2}$$

constrain: $p_1 + p_2 = 1$

$$p_1 = p_2 = \frac{1}{2}$$

$$f\left(\frac{1}{2}, \frac{1}{2}\right) = ln2 \approx 0.69$$

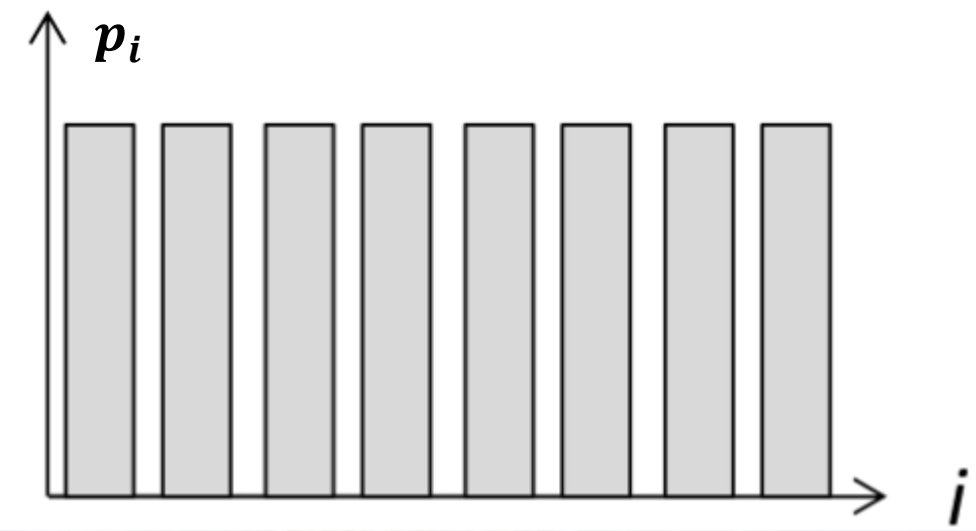
Lagrangian Multiplier
Examples

maximum entropy for I states:

$$f(p_1,..,p_i,...p_I) = -\sum_{i=1}^{I} p_i \, lnp_i$$

subject to

$$g(p_1,..,p_i,...p_I) = \sum_{i=1}^{I} p_i = 1$$



maximum subject to $g(p_1,..,p_i,...p_I)$:

$$\frac{\partial f(p_1,..,p_i,...p_I)}{\partial p_i} = \lambda \frac{\partial g(p_1,..,p_i,...p_I)}{\partial p_i}$$

$$-\ln p_i - 1 = \lambda \qquad p_i = e^{-(1+\lambda)}$$

constrain:

$$\sum_{i=1}^{I} e^{-(1+\lambda)} = 1$$

$$Ie^{-(1+\lambda)} = 1$$

$$e^{-(1+\lambda)} = \frac{1}{I}$$

probabilities are constant!
→ flat distribution!

$$p_i = \frac{1}{I}$$

maximum entropy for I states:

$$f(p_1,..,p_i,...p_I) = -\sum_{i=1}^{I} p_i \, lnp_i$$

subject to

$$g_1(p_1,..,p_i,...p_I) = \sum_{i=1}^{I} p_i = 1$$

if *N* and ***total energy*** is conserved

$$g_2(p_1,..,p_i,...p_I) = \sum_{i=1}^{I} p_i \varepsilon_i = \frac{E_{tot}}{N} = \frac{1}{N}\sum_{i=1}^{I} n_i \varepsilon_i$$

$$\frac{\partial f(p_1,..,p_i,...p_I)}{\partial p_i} = \lambda_1 \frac{\partial g_1(p_1,..,p_i,...p_I)}{\partial p_i} + \lambda_2 \frac{\partial g_2(p_1,..,p_i,...p_I)}{\partial p_i}$$

$$-\ln p_i - 1 = \lambda_1 + \lambda_2 \frac{\partial \sum_{j=1}^{I} p_j \varepsilon_j}{\partial p_i}$$

N:	number of indistinguishable particles
n_i:	number of particles in micro state *i*
I:	number of states
p_i:	probability of a particle being in micro state *I*
ε_i:	energy in state *i*

Avoiding Overfitting and Regularization (L1 & L2)

maximum entropy for I states:

$$f(p_1,..,p_i,...p_I) = -\sum_{i=1}^{I} p_i \, ln p_i$$

subject to

$$g_1(p_1,..,p_i,...p_I) = \sum_{i=1}^{I} p_i = 1$$

if *N* and ***total energy*** is conserved

$$g_2(p_1,..,p_i,...p_I) = \sum_{i=1}^{I} p_i \varepsilon_i$$

$$-\ln p_i - 1 = \lambda_1 + \lambda_2 \frac{\partial \sum_{j=1}^{I} p_j \varepsilon_j}{\partial p_i}$$

$$-\ln p_i - 1 = \lambda_1 + \lambda_2 \varepsilon_i$$

$$p_i = e^{-(1+\lambda_1)} \, e^{-\lambda_2 \varepsilon_i}$$

N:	number of indistinguishable particles
n_i:	number of particles in micro state *i*
I:	number of states
p_i:	probability of a particle being in micro state *I*
ε_i:	energy in state *i*

from g_1:

$$p_i = \frac{1}{\sum_{i=1}^{I} e^{-\lambda_2 \varepsilon_i}} \, e^{-\lambda_2 \varepsilon_i}$$

partition function *Z*

$$Z = \sum_{i=1}^{I} e^{-\lambda_2 \varepsilon_i}$$

Boltzmann distribution

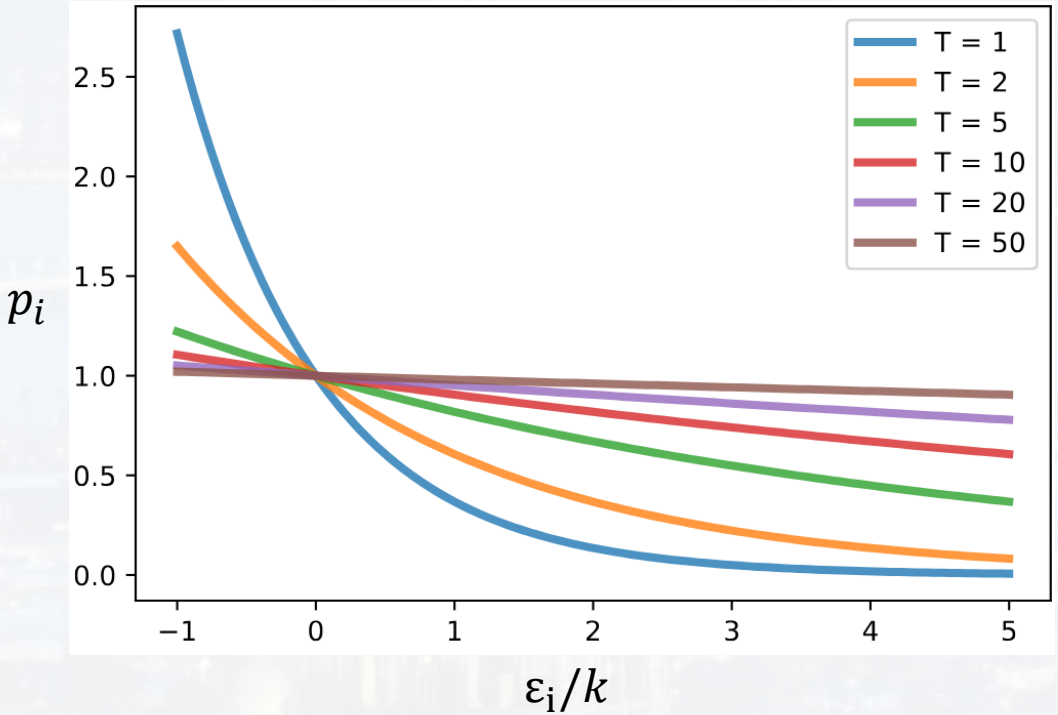
$$p_i = \frac{1}{Z} \, e^{-\lambda_2 \varepsilon_i}$$

Avoiding Overfitting and Regularization (L1 & L2)

partition function Z $$Z = \sum_{i=1}^{I} e^{-\lambda_2 \varepsilon_i}$$

Boltzmann distribution $$p_i = \frac{1}{Z} e^{-\lambda_2 \varepsilon_i}$$

one can show: $$\lambda_2 = \frac{1}{kT}$$



Lagrangian Multiplier
Examples

N:	number of indistinguishable particles
n_i:	number of particles in micro state *i*
I:	number of states
p_i:	probability of a particle being in micro state *I*
ε_i:	energy in state *i*

note:

- for $T \to \infty$, $Z \to I$, i. e. higher states become more accessible and $p_i \to \frac{1}{I}$
- we used maximum entropy: **equilibrium** state for large *N*
- *N* **and** E_{tot} are constant
- ANN: **softmax layer** for classification probabilities (see later)

Avoiding Overfitting and Regularization (L1 & L2)

Thank you very much for your attention!



